

Posicionamentos e Exibição de Elementos com CSS

CSS



Michele Queiroz Ambrosio

Desenvolvedora Front-end

@programi_

Flexbox

Michele Queiroz Ambrosio

Desenvolvedora Front-end

@programi_

Sobre Mim

Como entrei na área de tecnologia?

- (2012-2013)
Curso técnico em informática na ETEC;
- (2014-2017)
Faculdade de Ciência da Computação na UNISO;
- (2017-atualmente)
Desenvolvedora Front-end na Eduzz.

Sobre Mim

Como você pode me encontrar nas redes sociais?

- **Instagram:** @programi_
- **Twitch:** twitch.tv/michele_ambrosio
- **LinkedIn:** Michele Ambrosio
- **Github:** @micheleambrosio



Objetivo Geral

Ao longo dessa trilha você vai estar aprendendo:

- Fundamentos do CSS;
- Estilizações básicas com CSS;
- Unidades de Medida;
- Posicionamentos e displays;
- Alinhamento de elementos (flexbox e grid layout);
- Responsividade;

Objetivo Geral

Ao longo dessa trilha você vai estar aprendendo:

- Pseudo-elementos
- Pseudo-classes;
- Transições;
- Animações;
- Convenções no CSS e frameworks.

Pré-requisitos

É importante que você já tenha uma base de HTML e assistido ao módulo 1 de CSS, para acompanhar os exemplos e conseguir estilizar suas páginas.

Ferramentas utilizadas

- VSCode;
- Plugins para VSCode: Live Server e Emmet;
- Google Chrome.

Percurso

Etapa 1

Propriedades de exibição

Etapa 2

Posicionando os elementos

Hands On!

***“Falar é fácil.
Mostre-me o código!”***

Linus Torvalds



FLEXBOX



Flexbox

Michele Queiroz Ambrosio

Desenvolvedora front-end

@programi_

Sobre mim

(2012-2013)

Curso técnico em informática na ETEC;

(2014-2017)

Faculdade de Ciência da Computação na UNISO;

(2017-atualmente)

Desenvolvedora Front-end na Eduzz.

Sobre mim

Como você pode me encontrar
nas redes sociais?

Instagram: @programi_

Twitch: twitch.tv/michele_ambrosio

LinkedIn: Michele Ambrosio

Github: @micheleambrosio



Objetivo geral

O objetivo geral deste curso é ensinar os principais conceitos, aplicações e propriedades que envolvem o flexbox.

Pré-requisitos

É importante que você já tenha uma base de HTML e assistido ao **Módulo 1 de CSS**, juntamente com o curso de **Posicionamentos e Displays com CSS** para acompanhar os exemplos e conseguir estilizar suas páginas.

Ferramentas utilizadas

- VSCode;
- Plugins para VSCode: Live Server e Emmet;
- Google Chrome.

Percurso

Etapa 1

Entendendo os conceitos do flexbox

Etapa 2

Os eixos do flexbox

Etapa 3

Definindo a direção do flexbox com a propriedade flex-direction

Etapa 4

Quebra de linhas e colunas com a propriedade flex-wrap

Percurso

Etapa 5

Juntando a propriedade flex-direction e a flex-wrap em uma só propriedade abreviada (flex-flow)

Etapa 6

Alinhando os elementos no eixo principal com justify-content

Etapa 7

Alinhamento de elementos no eixo transversal com align-items

Etapa 8

Alinhando as linhas no container com align-content

Percurso

Etapa 9

Controlando os espaços entre os itens com gap

Etapa 10

Ordenando os itens com a propriedade order

Etapa 11

Propriedade flex-grow

Etapa 12

Propriedade flex-shrink

Percurso

Etapa 13

Propriedade flex-basis

Etapa 14

Shorthand flex

Etapa 15

Alinhando itens individualmente com align-self

Etapa 16

Materiais de Apoio

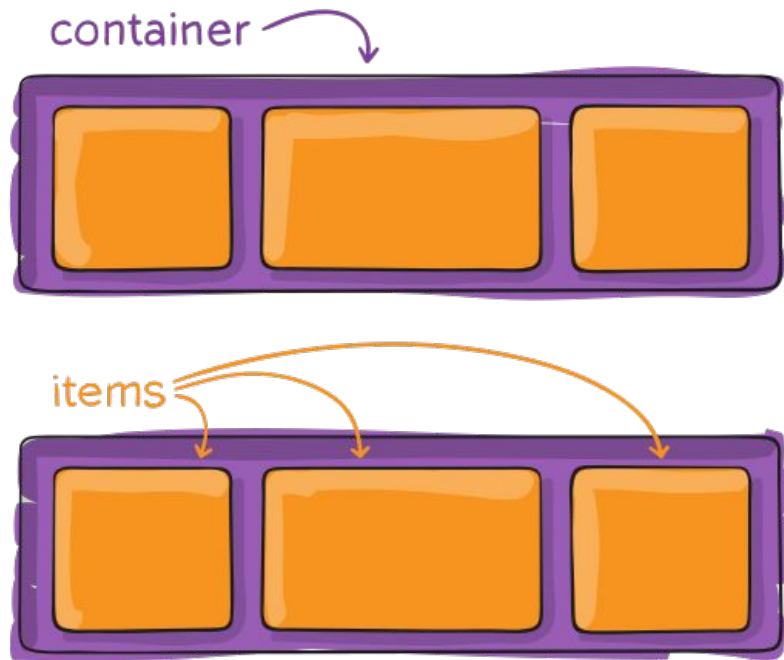
Etapa 1

Conceitos do flexbox

Conceitos do flexbox

Permite posicionar os elementos dentro de outro elemento (*container*);

Os elementos dentro do *container* são conhecidos como itens.



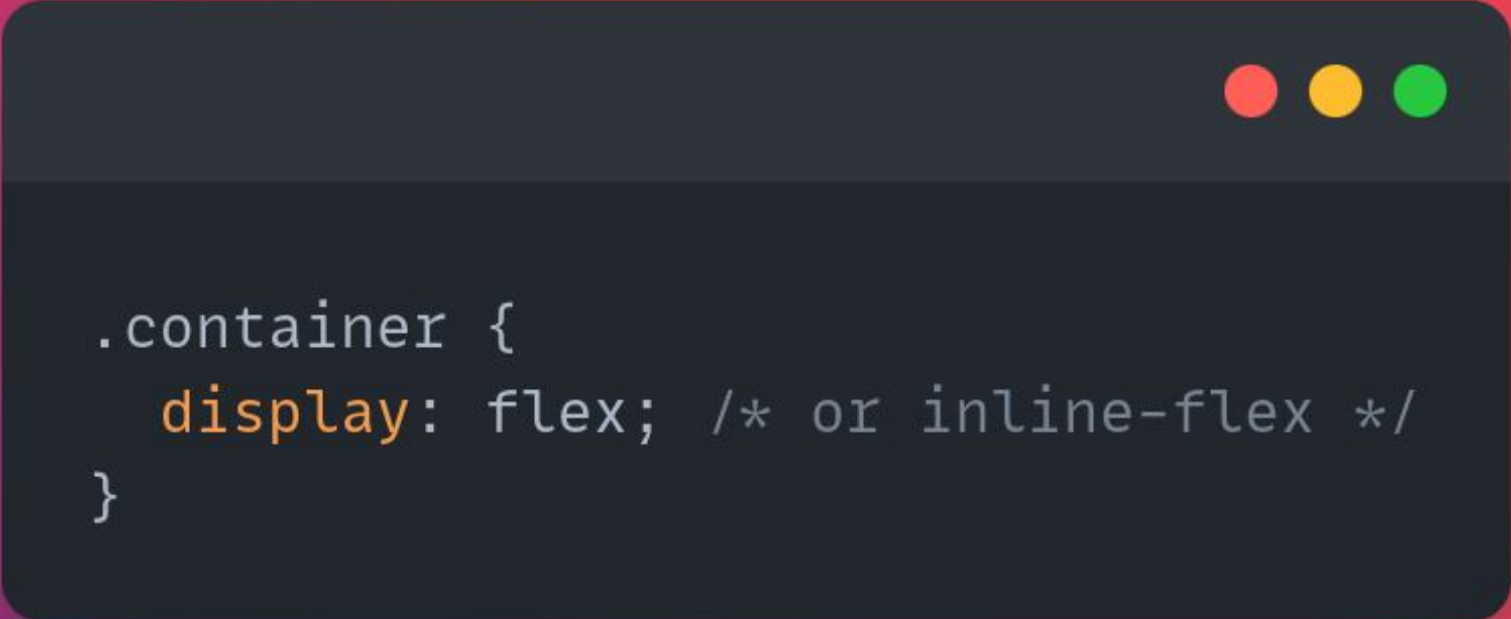
Conceitos do flexbox

O *flexbox* (*Flexible Box Module*) oferece uma maneira mais eficiente de organizar, alinhar e distribuir o espaço entre os itens do *container*, mesmo que o tamanho do elemento seja dinâmico.

Conceitos do flexbox

Permite trazer para o *container* a possibilidade de alterar a largura/altura; ordem dos itens e aproveitar o espaço disponível dentro do elemento da melhor forma.



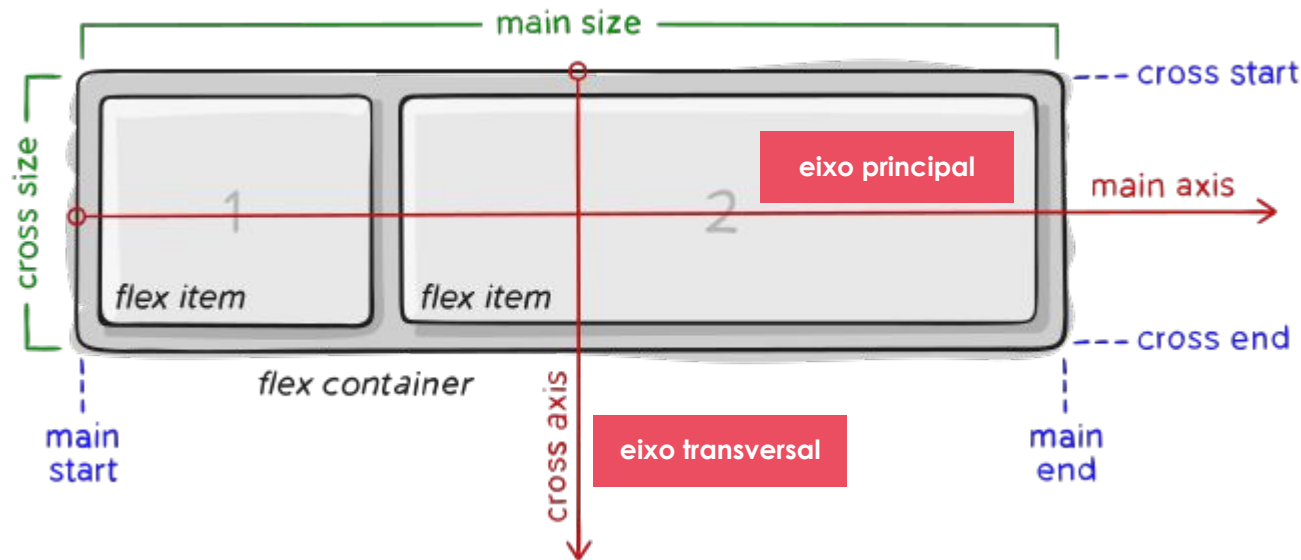


```
.container {  
  display: flex; /* or inline-flex */  
}
```

Etapa 2

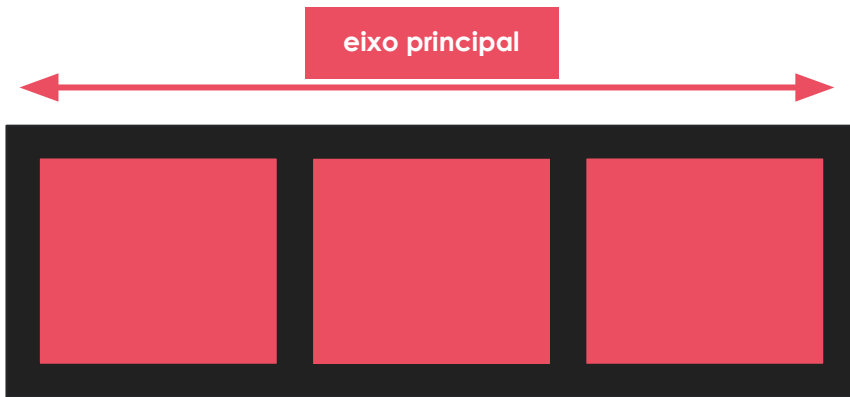
Os eixos do flexbox

Os eixos do flexbox



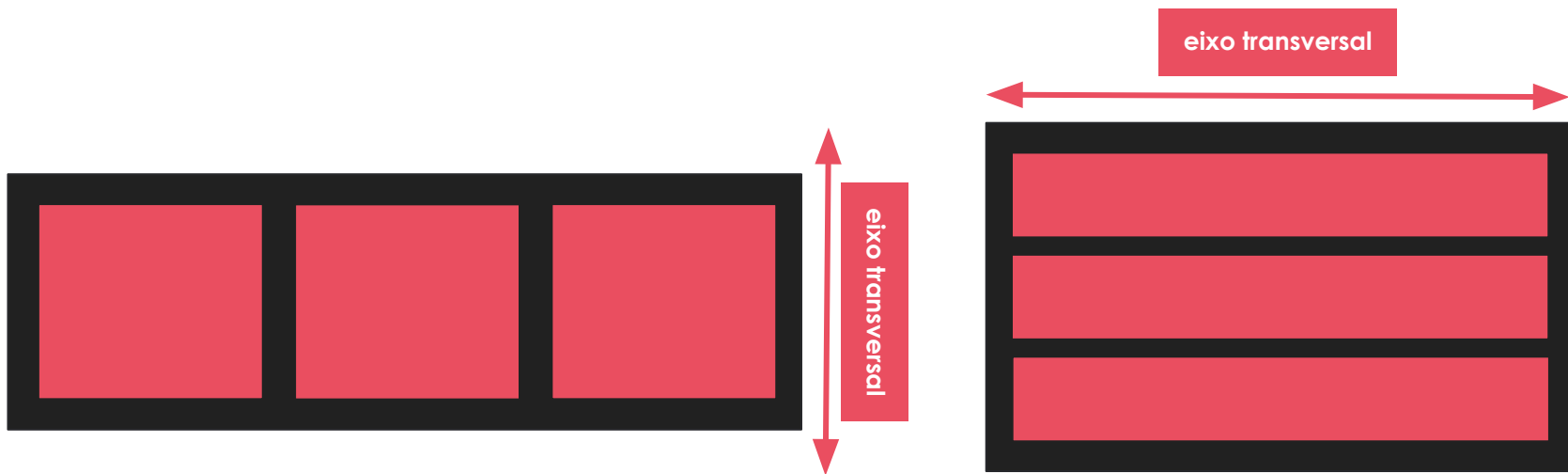
Eixo principal (main axis)

Define a direção em que o nosso container vai organizar os nossos itens: horizontalmente (em linha), ou verticalmente (em coluna).



Eixo transversal (cross axis)

O eixo transversal é perpendicular (atravessa) ao eixo principal.



Etapa 3

**Definindo a direção do
flexbox com a propriedade
flex-direction**

Propriedade flex-direction

flex-direction: row



flex-direction: row-reverse



flex-direction: column

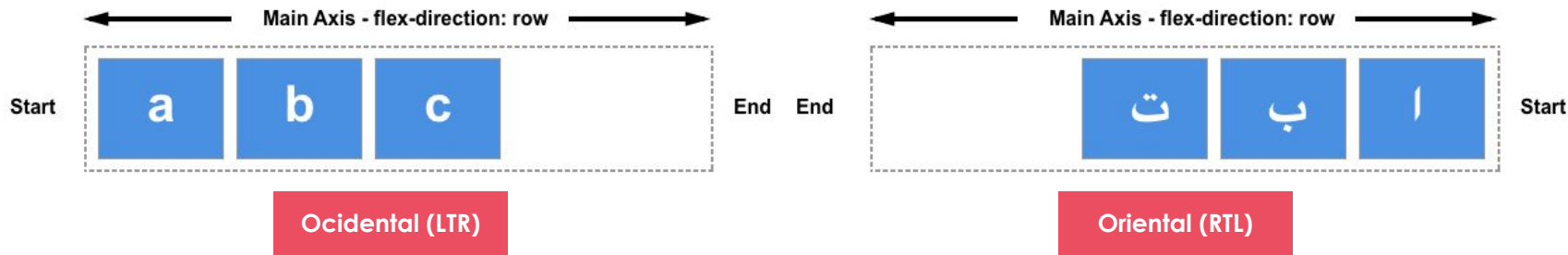


flex-direction:
column-reverse



Estilo da escrita & flexbox

A propriedade **flex-direction** vai levar em consideração o estilo de escrita no qual estamos trabalhando no projeto para definir se, **flex-direction: row**, por exemplo, deve ser da esquerda para direita, ou da direita para a esquerda.



Etapa 6

**Alinhando os elementos no
eixo principal com
justify-content**

Valores de justify-content

flex-start



flex-end



center



space-between



space-around

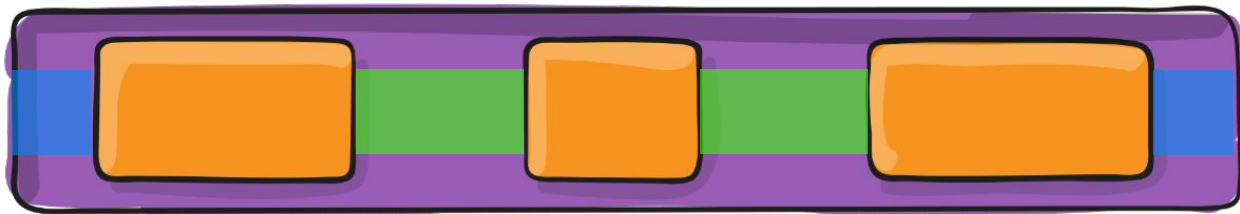


space-evenly

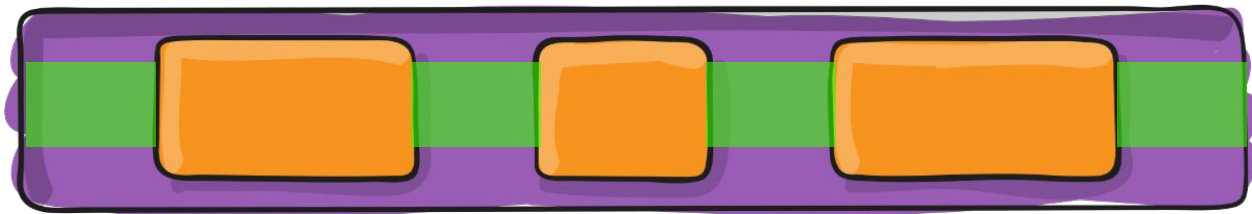


space-around x space-evenly

space-around



space-evenly

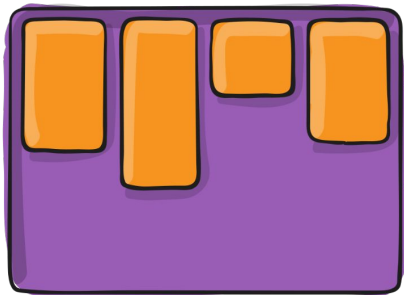


Etapa 7

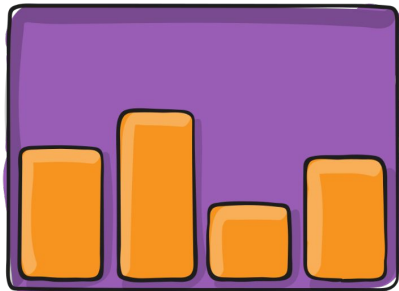
**Alinhando os elementos no
eixo transversal com
align-items**

Valores de align-items

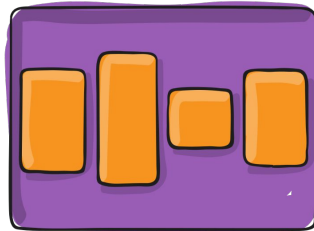
flex-start



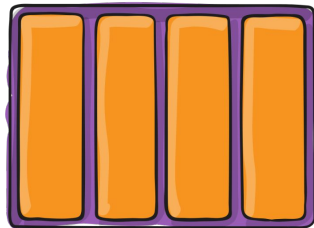
flex-end



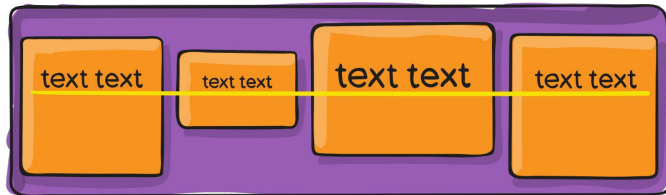
center



stretch



baseline

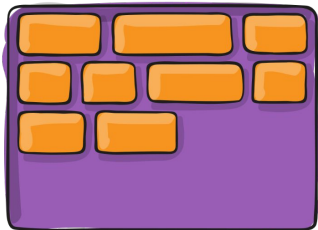


Etapa 8

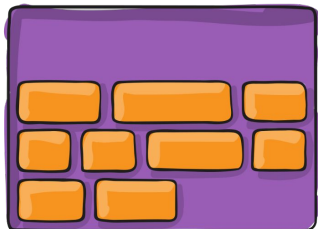
**Alinhando as linhas no
container com align-content**

Valores de align-content

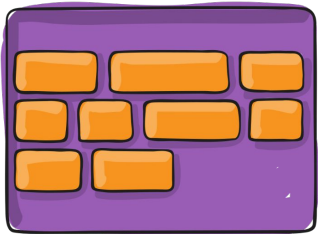
flex-start



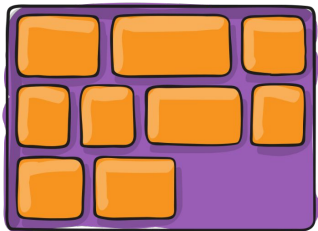
flex-end



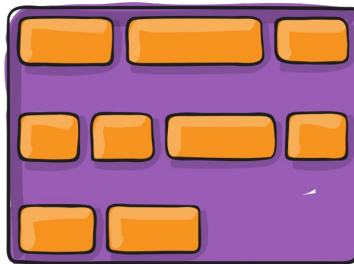
center



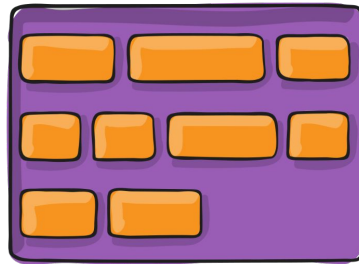
stretch



space-between



space-around



Etapa 9

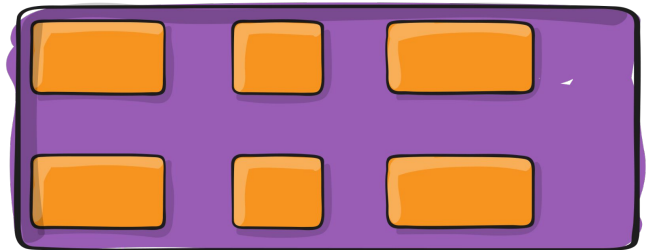
Controlando os espaços
entre itens com gap

Valores de gap

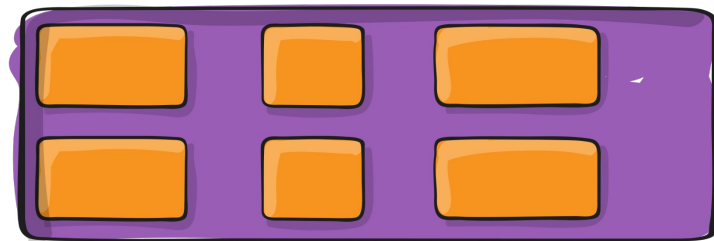
gap: 10px



gap: 30px



gap: 10px 30px





Etapa 10

Ordenando os itens do *container*

Ordenando os itens no container

Podemos ordenar os *flex-items* através da propriedade **order**.
Por padrão, os itens são exibidos conforme colocamos no HTML, mas, podemos ordená-lo individualmente.

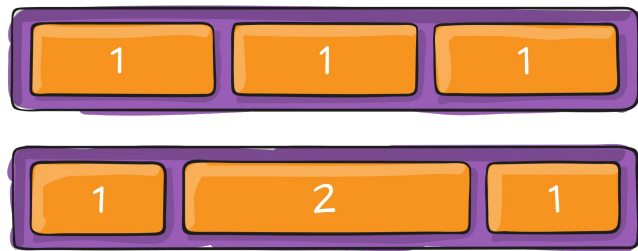


Etapa 11

Propriedade flex-grow

Propriedade flex-grow

A propriedade **flex-grow** fará com que esse espaço extra seja usado por esses itens, dando a eles a capacidade para o item de crescer, ao longo do eixo principal, caso for necessário.



Atenção: Números negativos não serão aceitos

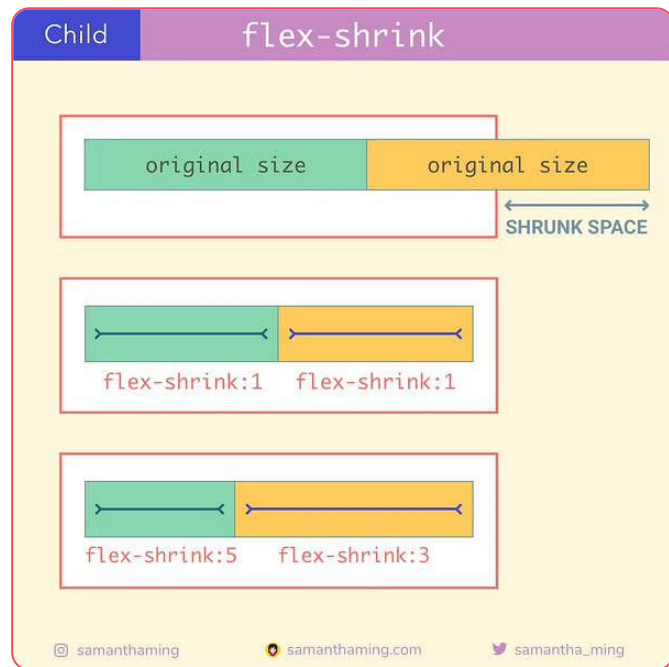
Etapa 12

Propriedade flex-shrink

Propriedade flex-shrink

A propriedade **flex-shrink** vai controlar o quanto esse item vai diminuir, caso seja necessário.

Se o *container* não possuir espaço suficiente para acomodar os elementos, eles irão diminuir para caber.



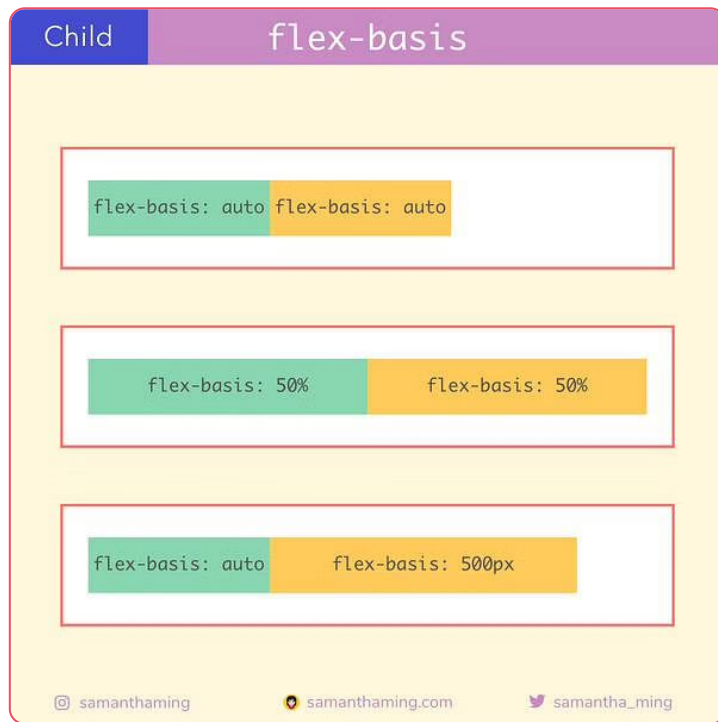
Etapa 13

Propriedade flex-basis

Propriedade flex-basis

A propriedade **flex-basis** define qual o tamanho inicial dos itens do nosso flexbox, antes que o espaço que de sobra seja redistribuído no *container*, de acordo com sua dimensão.

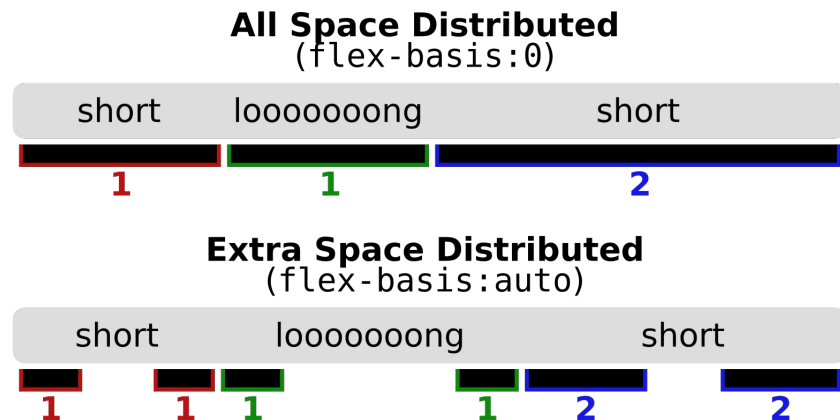
Pode ter um valor absoluto ou relativo para essa propriedade.



Flex-basis: distribuição do espaço

Caso seja aplicado

flex-basis: 0, o espaço extra em volta do conteúdo não será considerado. Se definirmos como **flex-basis: auto**, o espaço extra será distribuído com base no que definimos na propriedade **flex-grow**.



flex-basis x width/height

Se usarmos a propriedade **flex-basis** no item, ele irá sobrescrever o valor do **width/height** do elemento.

Porém, vai respeitar o **max-width/max-height** definido para o elemento, assim como o **min-width/min-height**.

flex-basis vs widths

<pre>.child { ✓ flex-basis: 500px width: 100px }</pre>	flex-basis wins 500px
<pre>.child { ✓ min-width: 100px flex-basis: 500px }</pre>	min-width wins 100px
<pre>.child { ✓ max-width: 700px flex-basis: 500px }</pre>	max-width wins 700px

© samanthaming samanthaming.com samantha_ming

Etapa 14

Propriedade flex (*shorthand*)

Propriedade flex (*shorthand*)

Normalmente, as propriedades **flex-grow**, **flex-shrink** e **flex-basis** não são usadas de forma individual, mas sim através da propriedade abreviada **flex**.

Os valores de **flex-grow**, **flex-shrink** e **flex-basis**, são definidos, respectivamente, através da propriedade **flex**.

Child

flex

`flex: flex-grow flex-shrink flex-basis`

`flex: flex-grow`
ONE VALUE, UNITLESS

`flex: flex-basis`
ONE VALUE: UNIT

`flex: flex-grow flex-basis`
TWO VALUES: UNITLESS & UNIT

`flex: flex-grow flex-shrink`
TWO VALUES: UNITLESS & UNITLESS

 samanthaming

 samanthaming.com

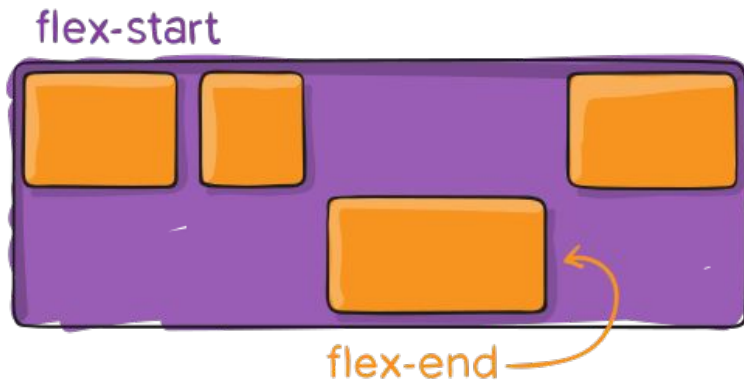
 samantha_ming

Etapa 15

**Alinhando os itens
individualmente com
align-self**

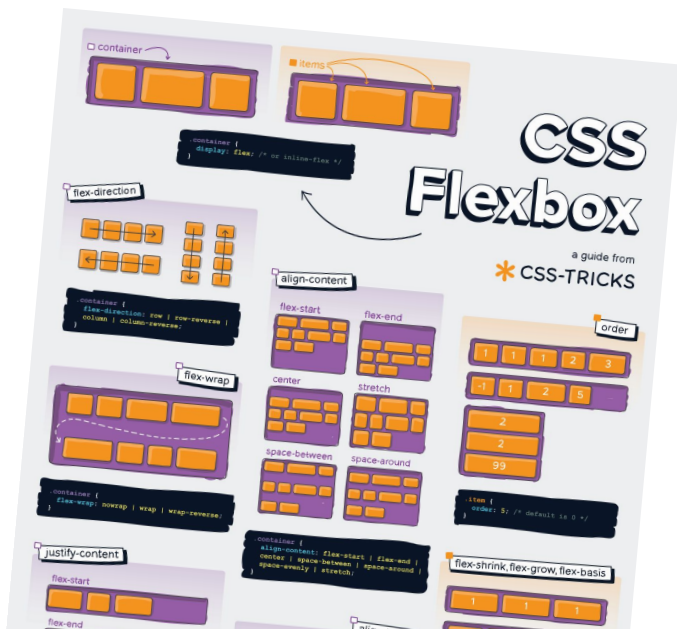
Alinhamento com align-self

A propriedade **align-self** permite que nós possamos sobrescrever o alinhamento padrão aplicado em todos os itens de forma geral que foi definido pela propriedade **align-items**.



Materiais de apoio

- 🔗 Flexbox Froggy
- 🔗 A Complete Guide to Flexbox
- 🔗 Flexbox - Guia Completo (Origamid)
- 🔗 Conceitos básicos de flexbox
- 🔗 Learn CSS - Flexbox





GRID LAYOUT



Grid Layout

Michele Queiroz Ambrosio

Desenvolvedora front-end

@programi_

Sobre mim

(2012-2013)

Curso técnico em informática na ETEC;

(2014-2017)

Faculdade de Ciência da Computação na UNISO;

(2017-atualmente)

Desenvolvedora Front-end na Eduzz.

Sobre mim

Como você pode me encontrar
nas redes sociais?

Instagram: @programi_

Twitch: twitch.tv/michele_ambrosio

LinkedIn: Michele Ambrosio

Github: @micheleambrosio



Objetivo geral

O objetivo geral deste curso é ensinar os principais conceitos, aplicações e propriedades que envolvem o grid layout.

Pré-requisitos

É importante que você já tenha uma base de HTML e assistido ao **Módulo 1 de CSS**, juntamente com o curso de **Posicionamentos e Displays com CSS** para acompanhar os exemplos e conseguir estilizar suas páginas.

Ferramentas utilizadas

- VSCode;
- Plugins para VSCode: Live Server e Emmet;
- Google Chrome.

Percurso

Etapa 1

Entendendo os conceitos do Grid Layout

Etapa 2

Nomenclaturas do Grid

Etapa 3

Iniciando com o CSS Grid

Etapa 4

Adicionando colunas e linhas ao Grid

Percurso

Etapas Etapa 5

Grid implícito e explícito

Etapas Etapa 6

Definindo o tamanho mínimo e máximo das faixas

Etapas Etapa 7

Alocando os itens do Grid nas posições específicas

Etapas Etapa 8

Áreas do Grid

Percurso

Etapa 9

Shorthand grid-template

Etapa 10

Definindo os espaçamentos dos elementos do Grid

Etapa 11

Shorthand grid

Etapa 12

Alinhando os itens com justify-items e align-items

Percurso

Etapa 13

Alinhando os itens da linha e coluna com `place-items`

Etapa 14

Alinhando o conteúdo do Grid com `justify-content` e `align-content`

Etapa 15

Alinhando o conteúdo do Grid horizontalmente e verticalmente com `place-content`

Etapa 16

Alinhando itens específicos com `justify-self` e `align-self`

Percurso

Etapa 17

Alinhando itens específicos horizontalmente e verticalmente com place-self

Etapa 18

Materiais de apoio

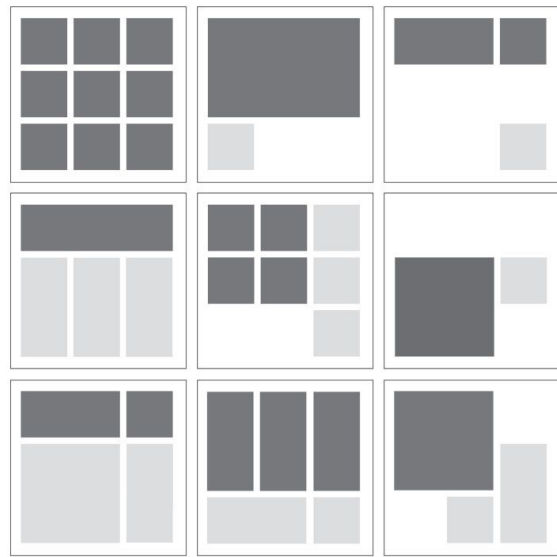


Etapa 1

Conceitos do grid layout

Entendendo os conceitos do grid

- Antigamente, era muito comum a construção de layouts usando tabelas, posicionamentos em bloco/linha e float;
- CSS Grid é um recurso que permite construir layouts baseados em grades bidimensionais, dividindo a página em regiões e definindo onde cada elemento deve ficar.

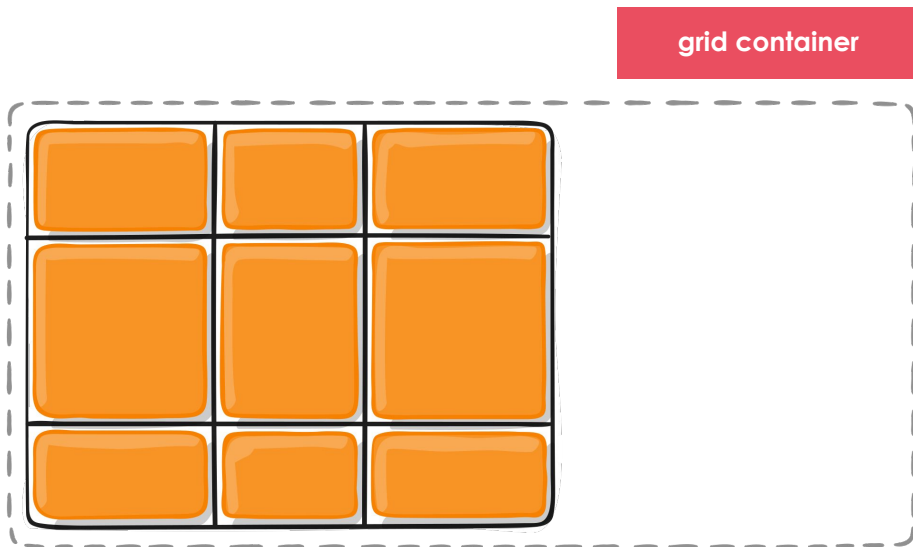


Etapa 2

Nomenclaturas do grid

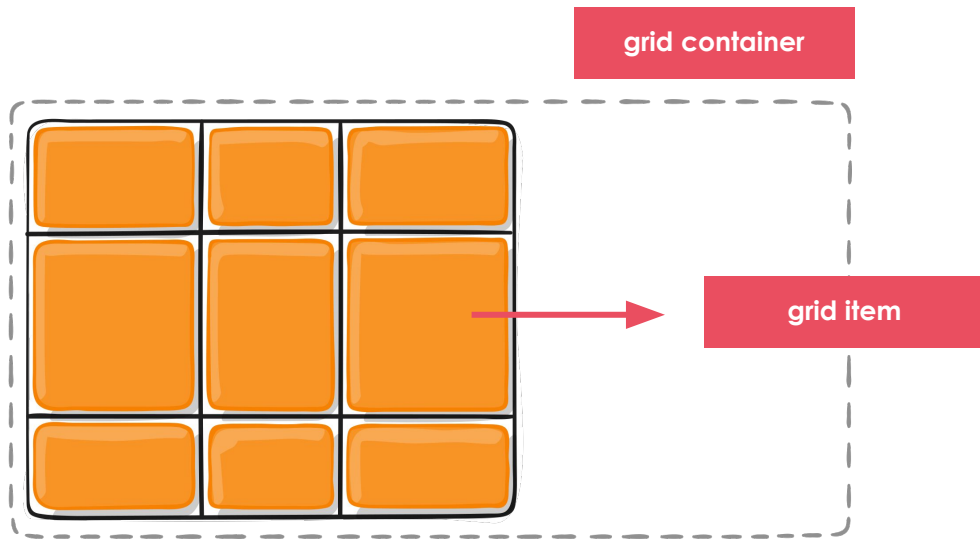
grid container

Elemento-pai que terá a propriedade **display: flex** especificada. Ele irá “envolver” todos os itens do grid.



grid item

Elementos que são filhos diretos do **grid container**.



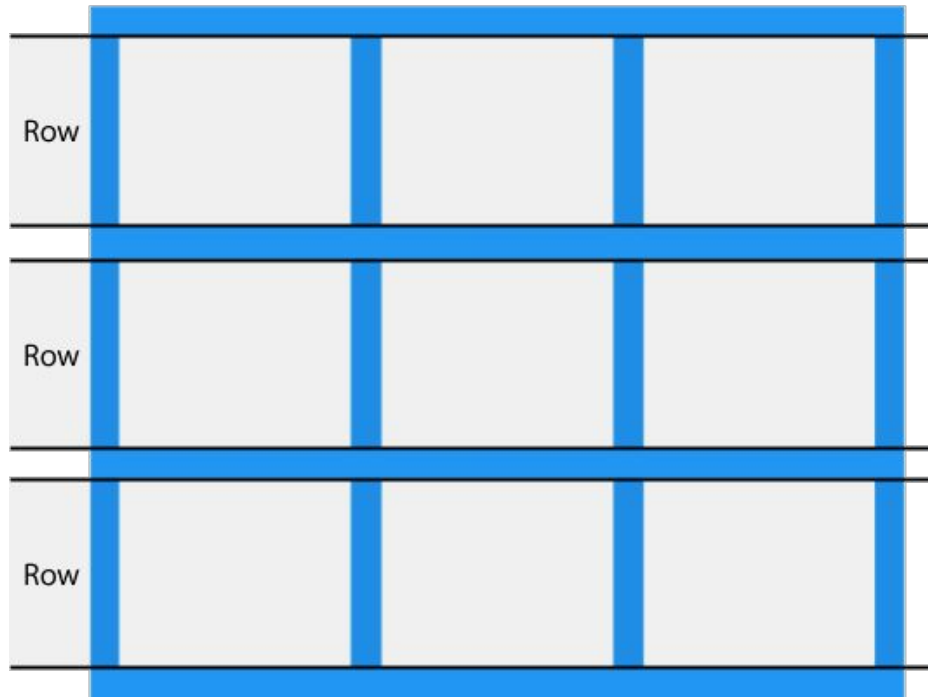
grid columns

As faixas verticais dos itens do grid, são chamados de colunas (*grid columns*)

Column	Column	Column

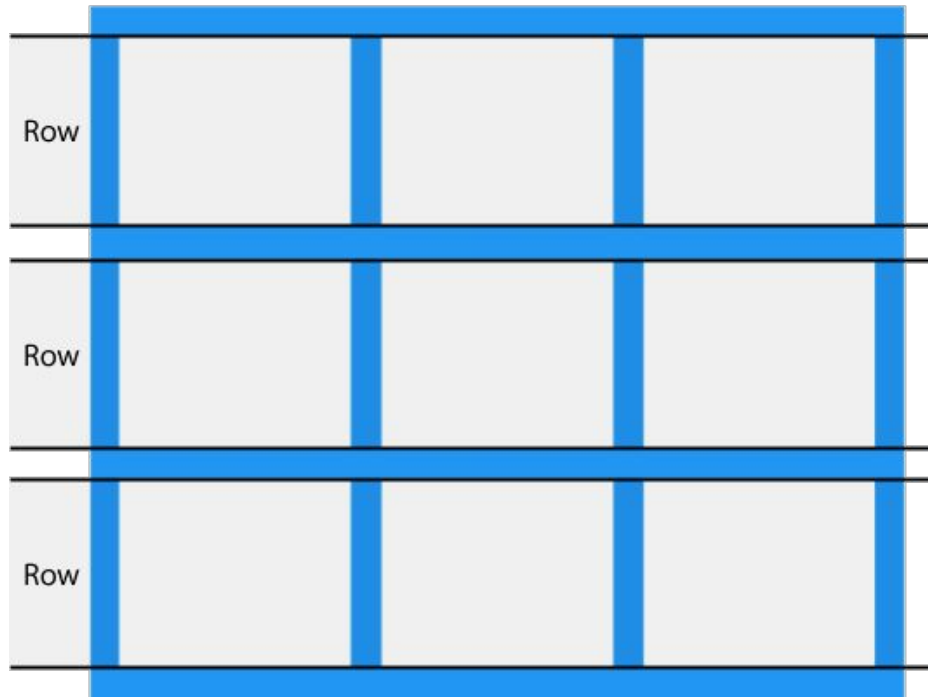
grid rows

As faixas horizontais dos itens do grid, são chamados de linhas (*grid rows*)



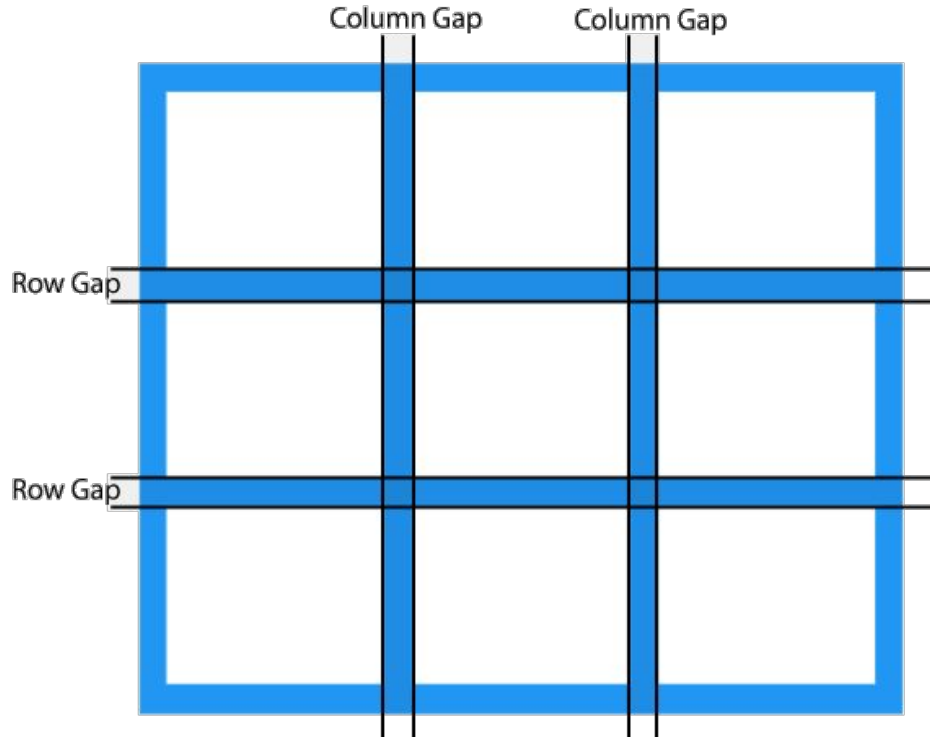
grid rows

As faixas horizontais dos itens do grid, são chamados de linhas (*grid rows*)



grid gaps

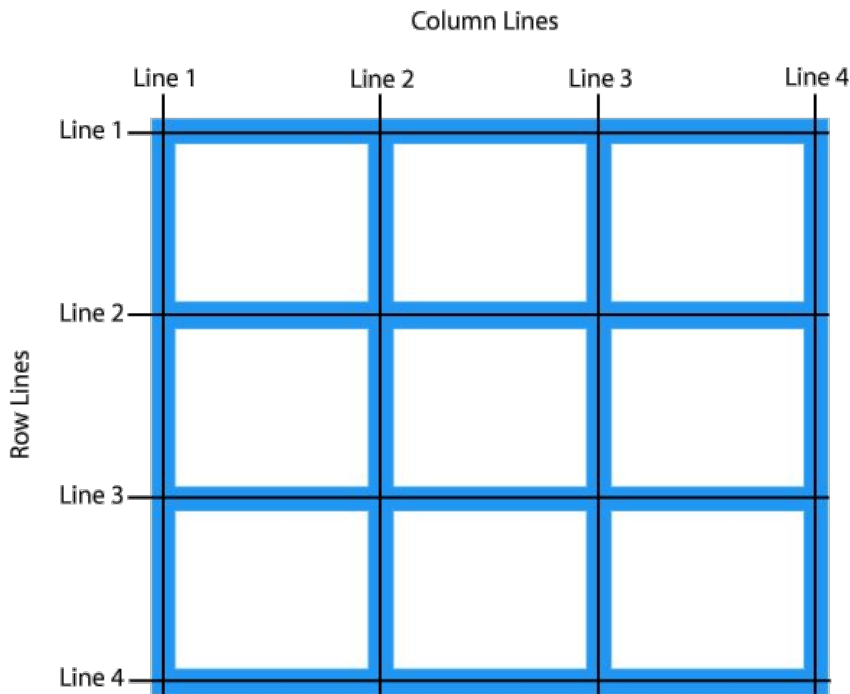
Espaços entre as linhas (*grid rows*) e coluna (*grid columns*) do grid.



grid line

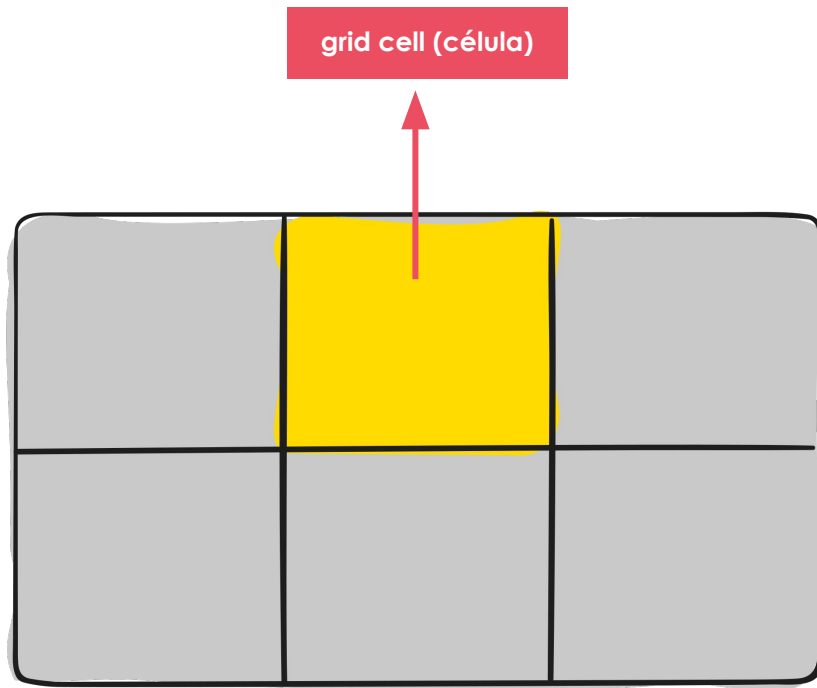
Divisórias que compõem a estrutura da grade.

As linhas entre as colunas são chamadas de *column lines* (linhas de coluna) e, as que estão entre as faixas horizontais, são chamadas de *row lines* (linhas de linha).



grid cell

São as células, ou seja, cada “quadrado” da grade, que vai entre uma *grid line* e outra.



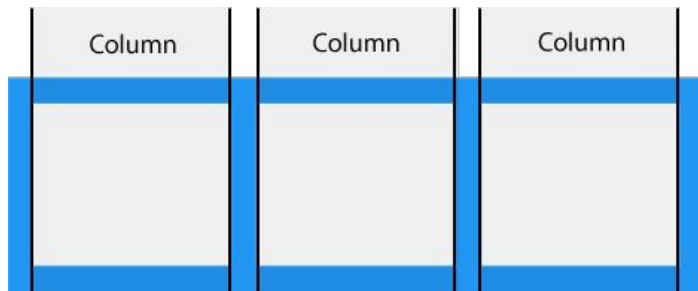
Etapa 4

Adicionando colunas e linhas ao grid

Linhas e colunas no grid

grid-template-columns

define as colunas (*columns*) do grid



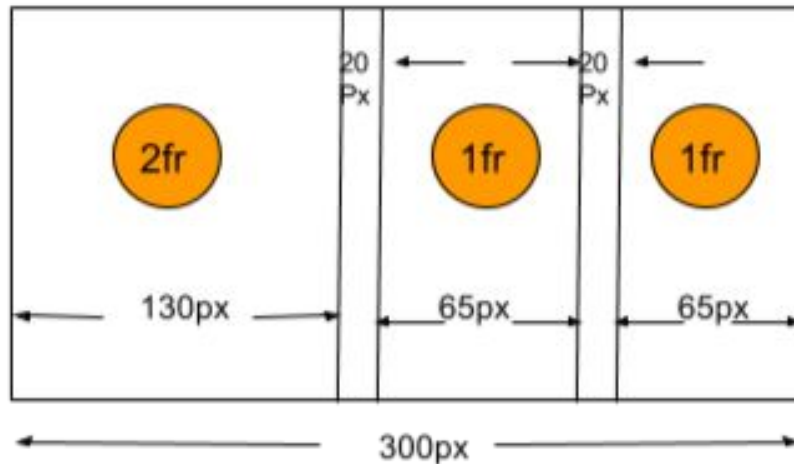
grid-template-rows

define as linhas (*rows*) do grid



Unidade de medida fr

Criada para o CSS Grid, a unidade de medida **fr** representa uma fração do espaço disponível no *container* do grid.



Função repeat

Permite repetir para todas, ou para algumas das faixas, o mesmo valor para seu tamanho de coluna ou linha.

```
/* Antes */  
.container {  
  grid-template-columns: 1fr 1fr 1fr;  
}  
  
/* Depois */  
.container {  
  grid-template-columns: repeat(3, 1fr);  
}
```

Função repeat: outros valores

auto-fill

```
grid-template-columns: repeat(auto-fill, minmax(100px, 1fr));
```



auto-fit

```
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
```



Etapa 5

Grid implícito e explícito

Unidade de medida fr

Quando define-se as colunas e linhas através das propriedades das propriedades **grid-template-columns** e **grid-template-rows**, estamos criando um grid explícito.

Mas, se os itens não couberem, o grid colocará um item fora desse grid que definimos, criando o que chamamos de grid implícito.

Propriedade grid-auto-flow

Controla como o algoritmo de reposicionamento automático irá se comportar.

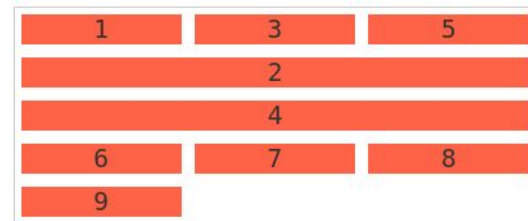
grid-auto-flow: row;



grid-auto-flow: column;



grid-auto-flow: dense;



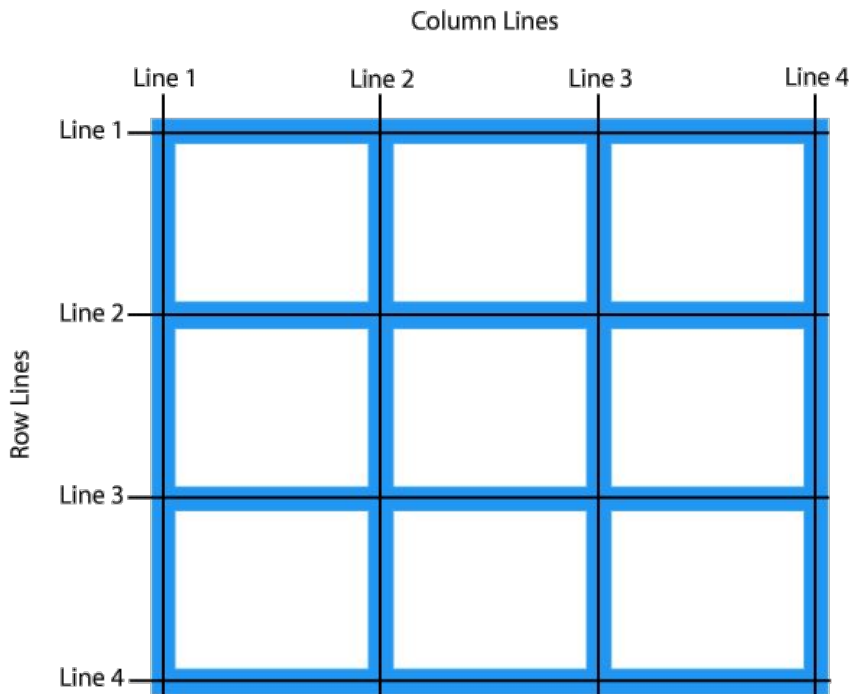
Etapa 7

**Alocando os itens do Grid nas
posições específicas**

Grid Lines

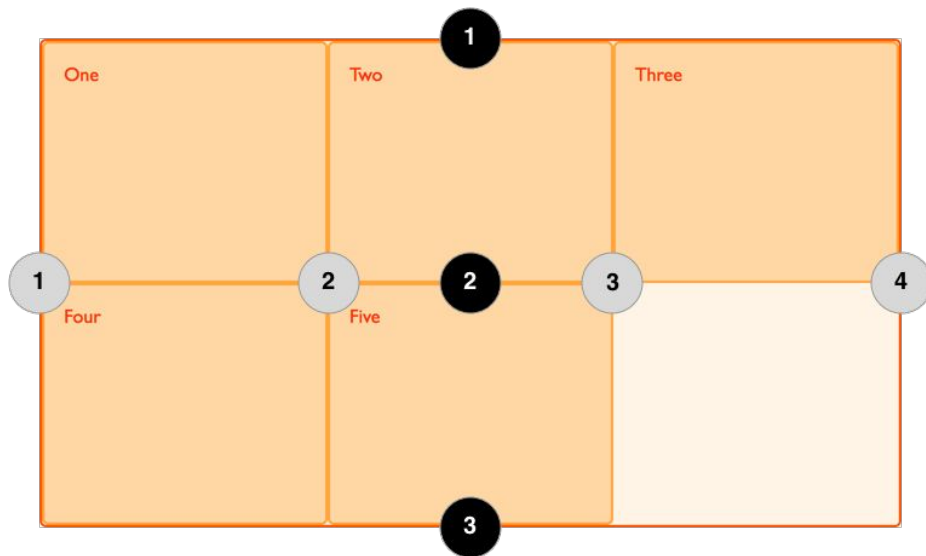
Divisórias que compõem a estrutura da grade.

As linhas entre as colunas são chamadas de *column lines* (linhas de coluna) e, as que estão entre as faixas horizontais, são chamadas de *row lines* (linhas de linha).

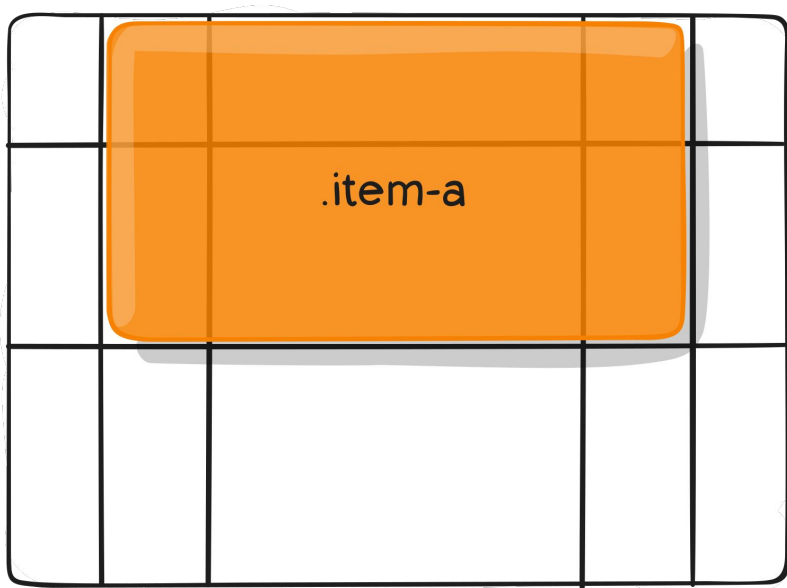


Grid Lines

Cada linha possui um número para que possamos usar de referência (exceto para grids implícitos).

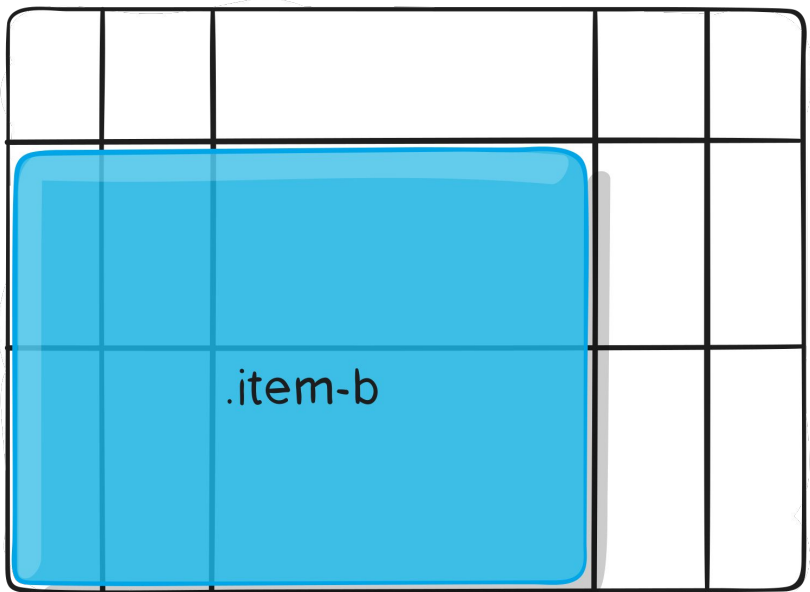


Posicionando os itens do grid



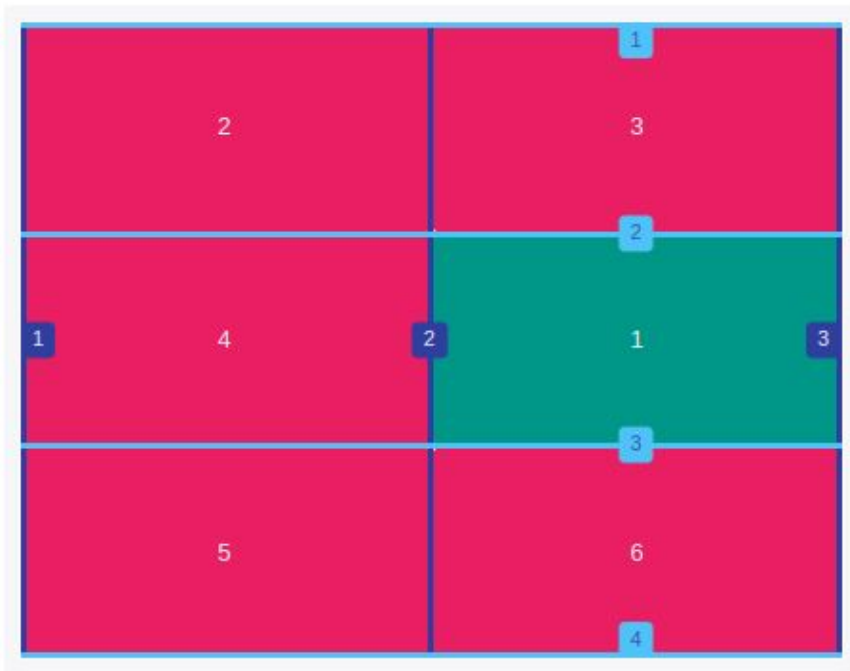
```
.item {  
  grid-column-start: 2;  
  grid-column-end: 5;  
}
```

Posicionando os itens do grid



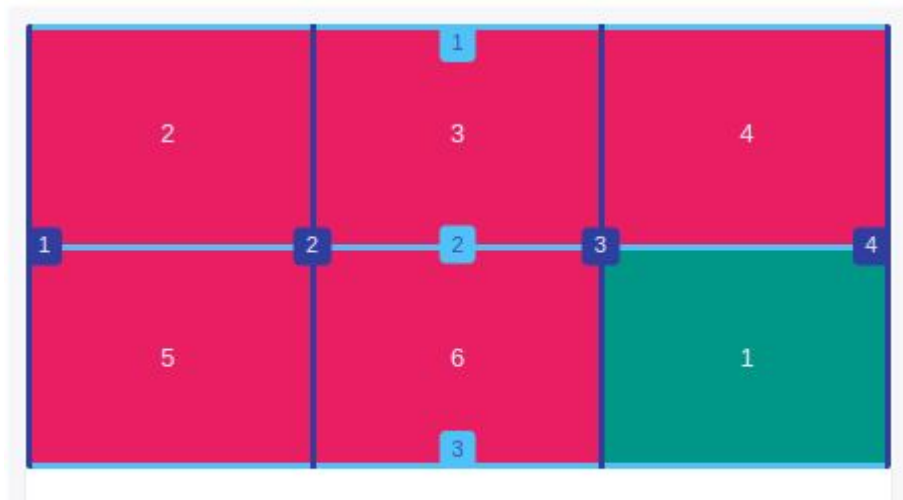
```
.item {  
  grid-column-start: 1;  
  grid-column-end: span 3;  
  grid-row-start: 2;  
  grid-row-end: span 2;  
}
```

Posicionando os itens do grid



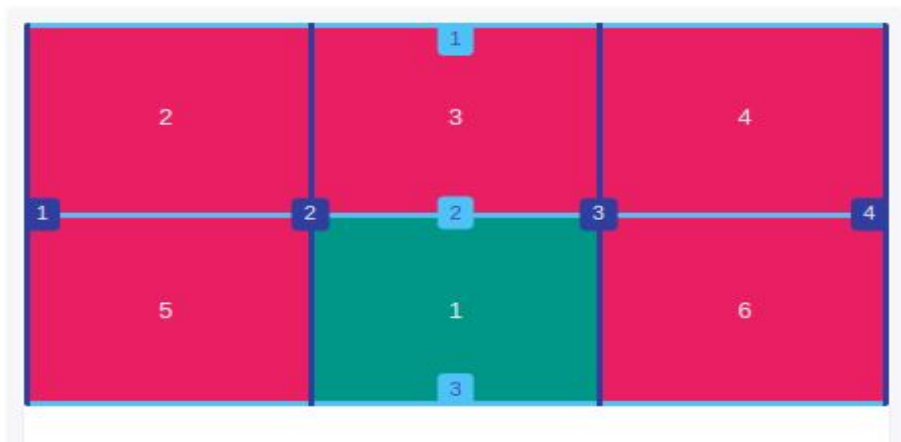
```
.item-1 {  
  grid-row-start: 2;  
  grid-row-end: 3;  
  grid-column-start: 2;  
  grid-column-end: 3;  
}
```

Posicionando os itens do grid



```
.item-1 {  
  grid-row-start: 2;  
  grid-row-end: 3;  
  grid-column-start: 2;  
  grid-column-end: 3;  
}
```

Posicionando os itens do grid



```
.item-1 {  
  grid-area: 2 / 2 / 3 / 3;  
}
```

Etapa 8

Áreas do grid

Shorthand de outras propriedades

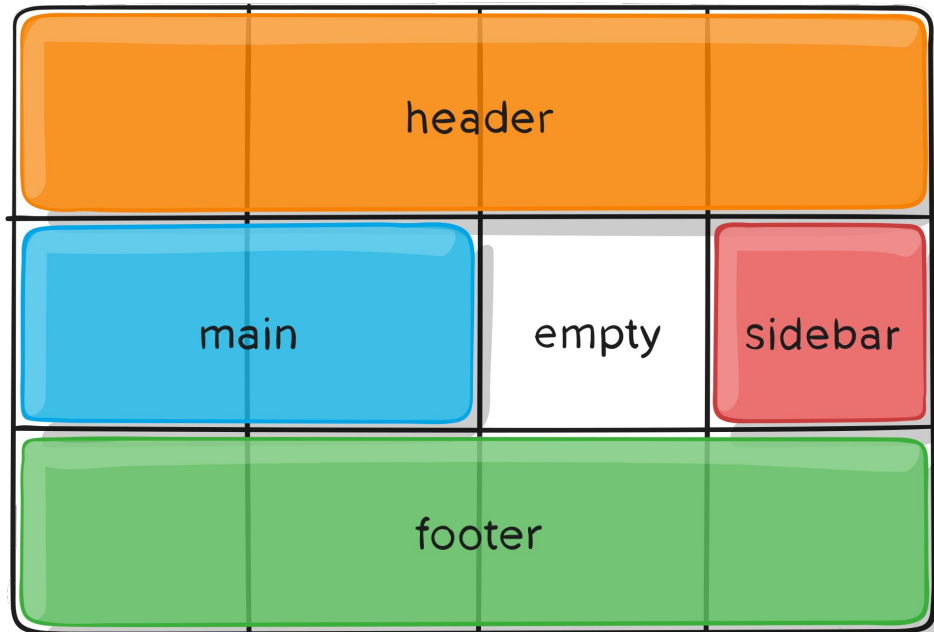
Através da propriedade **grid-area**, nós conseguimos definir os valores das propriedades **grid-row-start**, **grid-row-end**, **grid-column-start** e **grid-column-end** de uma só vez.

```
/* Antes */  
.item {  
  grid-column-start: segunda-linha-col;  
  grid-column-end: span 4;  
  
  grid-row-start: linha-dois;  
  grid-row-end: span 2;  
}  
  
/* Depois */  
.item {  
  grid-area: linha-dois / segunda-linha-col / span 2 / span 4;  
}|
```

grid-area: <grid-row-start>/<grid-column-start>/<grid-row-end>/<grid-column-end>

Nomeando as áreas

É possível nomear as áreas do grid e posicionar os itens dentro das áreas específicas, as referenciando através do nome.

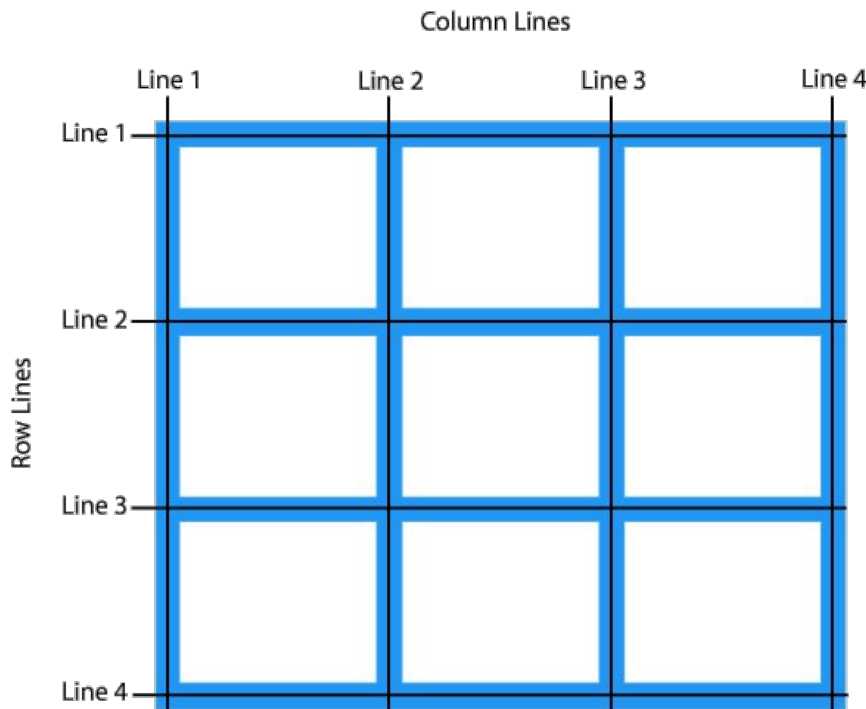


Etapa 10

Definindo os espaçamentos dos elementos do grid

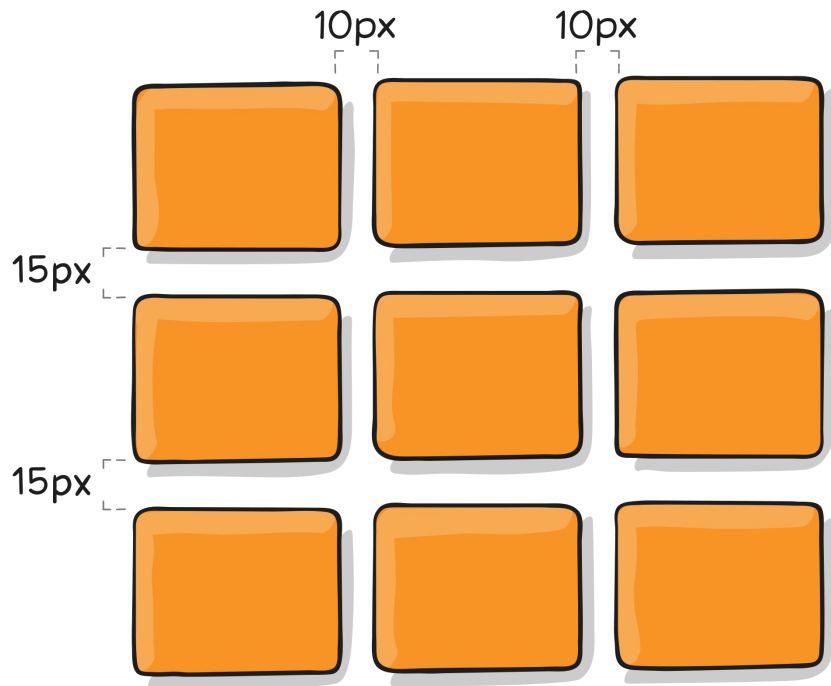
Espaçamento entre as faixas do grid

Tamanho das linhas do grid (*grid lines*) através das propriedades **column-gap** e **row-gap** (ou através da *shorthand gap*)



Espaçamento entre as faixas do grid

Os espaçamentos são adicionados apenas **entre** uma linha/coluna e outra.



Etapa 11

Shorthand “grid”

```
/* Antes */
```

```
.container {  
  grid-template-rows: 100px 300px;  
  grid-template-columns: 3fr 1fr;  
}
```

```
/* Depois */
```

```
.container {  
  grid: 100px 300px / 3fr 1fr;  
}
```

```
/* Antes */  
.container {  
  grid-auto-flow: row;  
  grid-template-columns: 200px 1fr;  
}
```

```
/* Depois */  
.container {  
  grid: auto-flow / 200px 1fr;  
}
```

```
/* Antes */
```

```
.container {  
  grid-auto-flow: row dense;  
  grid-auto-rows: 100px;  
  grid-template-columns: 1fr 2fr;  
}
```

```
/* Depois */
```

```
.container {  
  grid: auto-flow dense 100px / 1fr 2fr;  
}
```

```
/* Antes */
```

```
.container {  
  grid-template-rows: 100px 300px;  
  grid-auto-flow: column;  
  grid-auto-columns: 200px;  
}
```

```
/* Depois */
```

```
.container {  
  grid: 100px 300px / auto-flow 200px;  
}
```

```
/* Antes */  
.container {  
  grid-template-areas:  
    "header header header"  
    "footer footer footer";  
  grid-template-rows: 1fr 25px;  
  grid-template-columns: auto 50px auto;  
}
```

```
/* Depois */  
.container {  
  grid: "header header header" 1fr  
        "footer footer footer" 25px  
        / auto 50px auto;  
}
```

```
/* Gera uma linha com 100px de altura e 2 colunas com 1fr. */  
grid: 100px / 1fr 1fr
```

```
/* Gera uma linha com 100px de altura. O grid-auto-flow é definido como column  
(pois está logo antes da definição das colunas). Ele também define o  
grid-auto-columns com 100px 50px*/
```

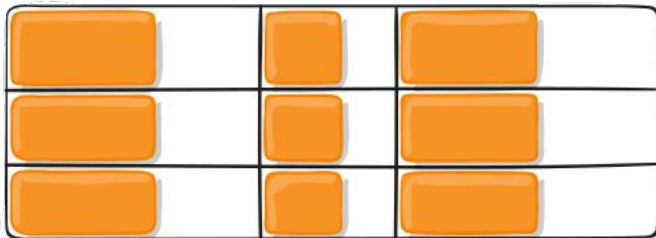
```
grid: 100px / auto-flow 100px 50px
```

Etapa 12

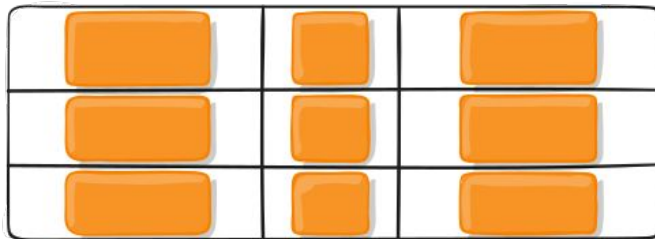
**Alinhando os itens com
justify-items e align-items**

Valores de justify-items

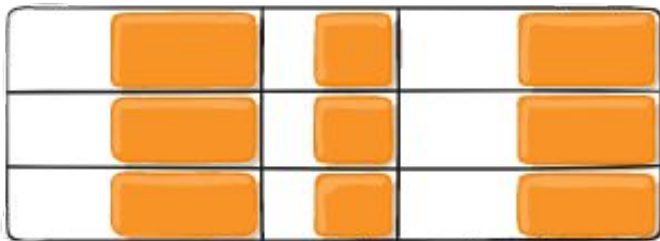
justify-items: start;



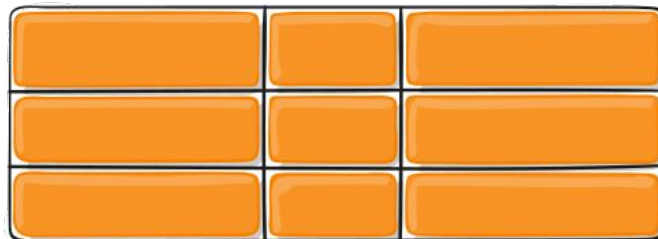
justify-items: center;



justify-items: end;

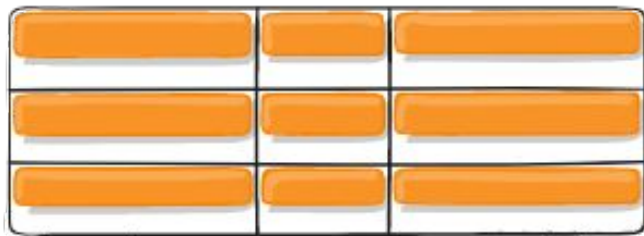


justify-items: stretch;

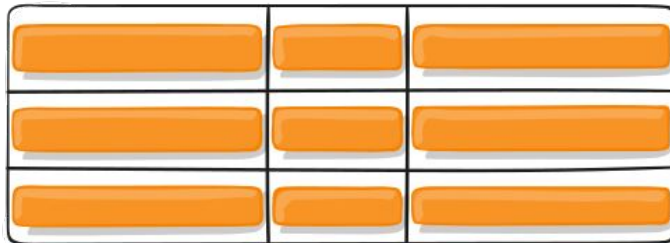


Valores de align-items

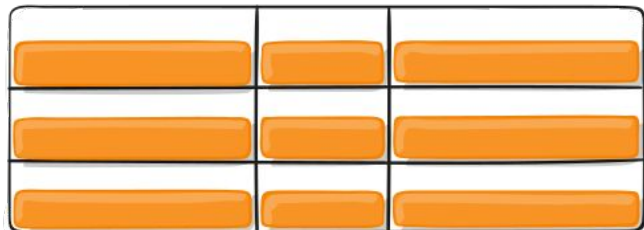
align-items: start;



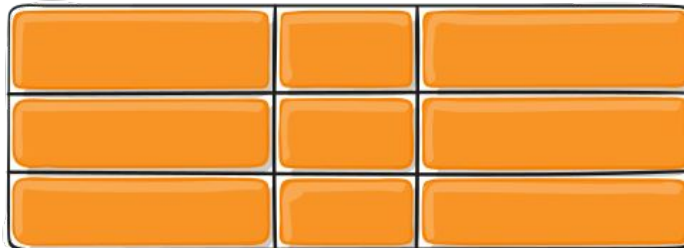
align-items: center;



align-items: end;



align-items: stretch;

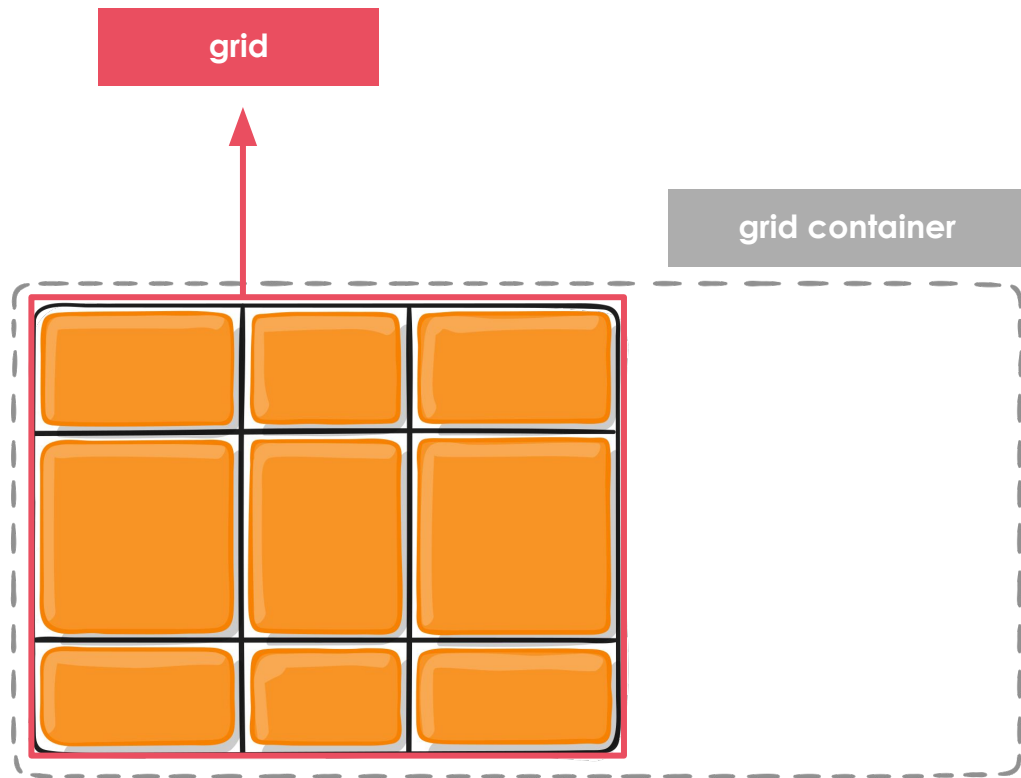


Etapa 14

**Alinhando o conteúdo do grid
com justify-content e
align-content**

Container do grid

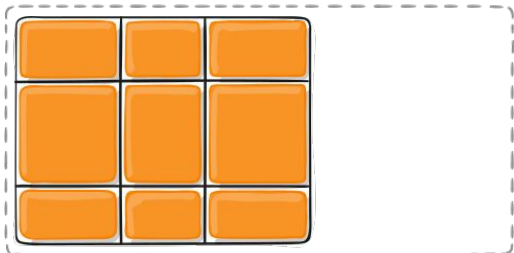
Container é o elemento que envolve a grade na qual estamos criando e, muitas vezes, ela acaba sendo maior do que o grid.



Valores de justify-content

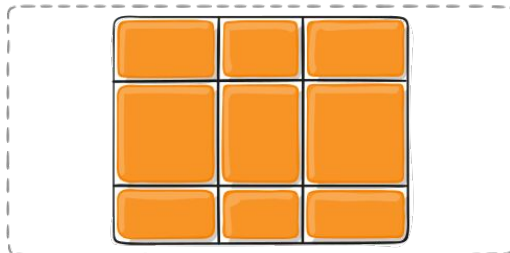
justify-content: start;

grid container



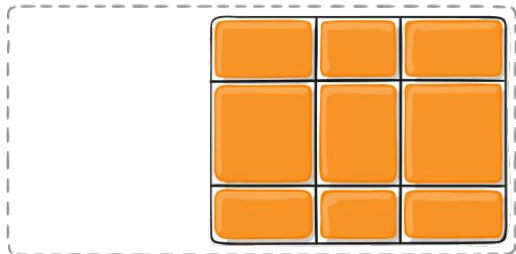
justify-content: center;

grid container



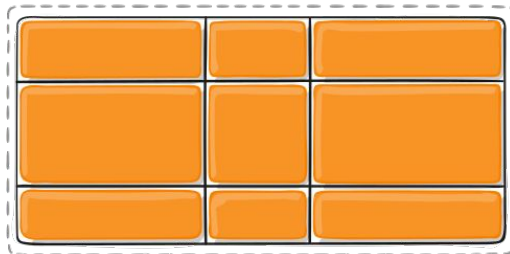
justify-content: end;

grid container



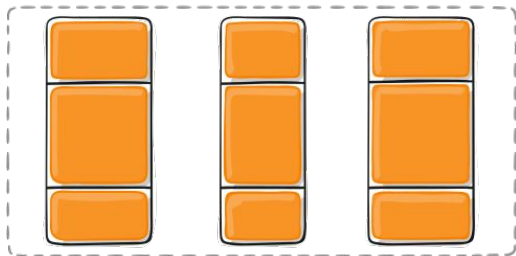
justify-content: stretch;

grid container

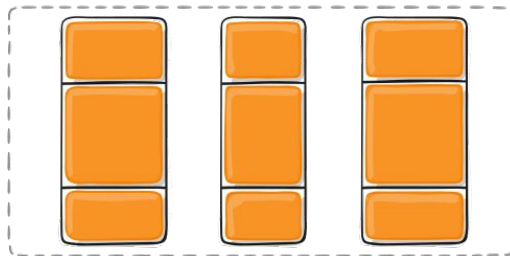


Valores de justify-content

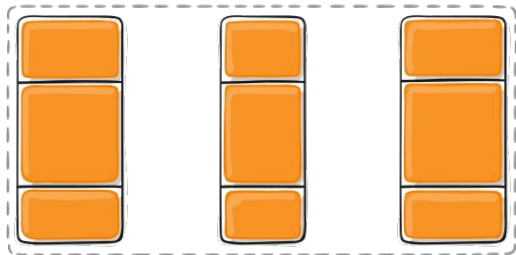
justify-content: space-around;
grid container



justify-content: space-evenly;
grid container



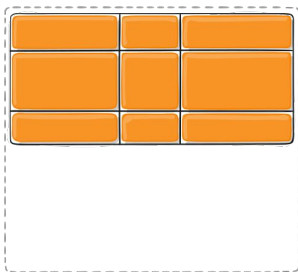
justify-content: space-between;
grid container



Valores de align-content

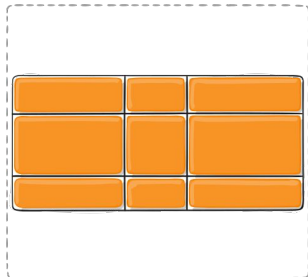
align-content: start;

grid container



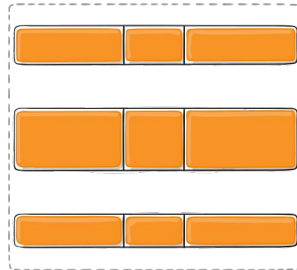
align-content: center;

grid container



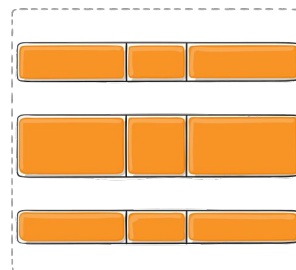
align-content: space-around;

grid container



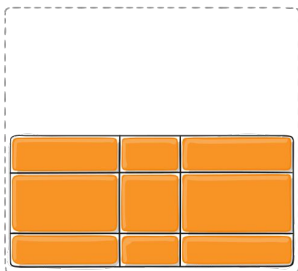
align-content: space-evenly;

grid container



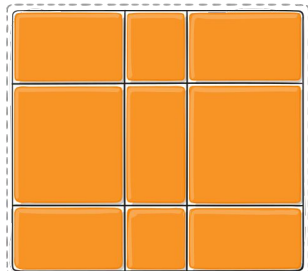
align-content: end;

grid container



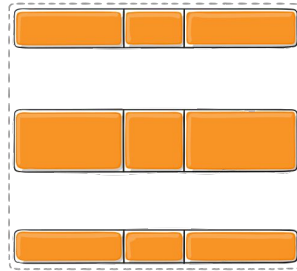
align-content: stretch;

grid container



align-content: space-between;

grid container



Etapa 16

**Alinhando itens específicos
com justify-self e align-self**

Valores de justify-self

justify-self: start;

.item-a		

justify-self: center;

.item-a		

justify-self: end;

	.item-a	

justify-self: stretch;

.item-a		

Valores de align-self

align-self: start;

.item-a		

align-self: center;

.item-a		

align-self: end;

.item-a		

align-self: stretch;

.item-a		

Materiais de apoio

- 🔗 [Grid Garden](#)
- 🔗 [CSS Grid Layout Module - W3Schools](#)
- 🔗 [A Complete Guide to Grid](#)
- 🔗 [CSS Grid Layout](#)
- 🔗 [Grid Guide \(Origamid\)](#)

RESPONSIVIDADE



Responsividade

Michele Queiroz Ambrosio

Desenvolvedora front-end

@programi_

Sobre mim

(2012-2013)

Curso técnico em informática na ETEC;

(2014-2017)

Faculdade de Ciência da Computação na UNISO;

(2017-atualmente)

Desenvolvedora Front-end na Eduzz.

Sobre mim

Como você pode me encontrar nas redes sociais?

Instagram: @programi_

Twitch: twitch.tv/michele_ambrosio

LinkedIn: Michele Ambrosio

Github: @micheleambrosio



Objetivo geral

O objetivo geral deste curso é mostrar os conceitos de responsividade e ensinar as técnicas que podemos utilizar para deixarmos nosso site responsivo.

Pré-requisitos

É importante que você já tenha uma base de HTML e assistido ao **Módulo 1 de CSS**, juntamente com os cursos de **Posicionamentos e Displays com CSS, Flexbox e Grid Layout** para acompanhar os exemplos e conseguir estilizar suas páginas.

Ferramentas utilizadas

- VSCode;
- Plugins para VSCode: Live Server e Emmet;
- Google Chrome.

Percorso

Etapa 1

Introdução ao Design Responsivo

Etapa 2

Layouts flexíveis - Multicolunas

Etapa 3

Layouts flexíveis - Flexbox

Etapa 4

Layouts flexíveis - Grid

Percurso

Etapa 5

Meta tag viewport

Etapa 6

O que são Media Queries?

Etapa 7

Estrutura das Media Queries

Etapa 8

Media Types (tipos de mídia)

Percurso

Etapa 9

Media Features (recursos de mídia)

Etapa 10

Operadores Lógicos

Etapa 11

Maneiras de usar as Media Queries

Etapa 12

Breakpoints

Percurso

Etapa 13

Devemos sempre usar as media queries?

Etapa 14

Imagens responsivas

Etapa 15

Tipografia responsiva

Etapa 16

Mobile first



Etapa 1

Introdução ao design responsivo

Técnicas para responsividade

(Ethan Marcotte)

1. Ideia de grids fluídos;
2. Técnica de imagens fluídas;
3. Media Queries.



Design Responsivo

Conjunto de melhores práticas para que um layout **responda** ao dispositivo que está sendo usado para visualizar uma página web.



Etapa 2

Layouts flexíveis

Layouts flexíveis

Uma das maneiras de se construir layouts flexíveis é usando o Grid Layout; Flexbox e Múltiplas Colunas, combinado com unidades de medidas relativas, incluindo as relacionadas à *viewport*.



Multicolunas (*multicol*)

Especifica-se em quantas colunas o conteúdo será dividido e o navegador irá calcular o tamanho delas conforme o tamanho da tela mudar.

Simple multicol example

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci, pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at ultricies tellus laoreet sit amet. Sed auctor

cursus massa at porta. Integer ligula ipsum, tristique sit amet orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat. Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.

Nam vulputate diam nec tempor bibendum. Donec luctus augue

eget malesuada ultrices. Phasellus turpis est, posuere sit amet dapibus ut, facilisis sed est. Nam id risus quis ante semper consectetur eget aliquam lorem. Vivamus tristique elit dolor, sed pretium metus suscipit vel. Mauris ultricies lectus sed lobortis finibus. Vivamus eu urna eget velit cursus viverra quis vestibulum sem. Aliquam tincidunt eget purus in interdum. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

monitor

Simple multicol example

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci, pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at ultricies tellus laoreet sit amet. Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat. Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra

tablet

congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.

Nam vulputate diam nec tempor bibendum. Donec luctus augue eget malesuada ultrices. Phasellus turpis est, posuere sit amet dapibus ut, facilisis sed est. Nam id risus quis ante semper consectetur eget aliquam lorem. Vivamus tristique elit dolor, sed pretium metus suscipit vel. Mauris ultricies lectus sed lobortis finibus. Vivamus eu urna eget velit cursus viverra quis vestibulum sem. Aliquam tincidunt eget purus in interdum. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

Simple multicol example

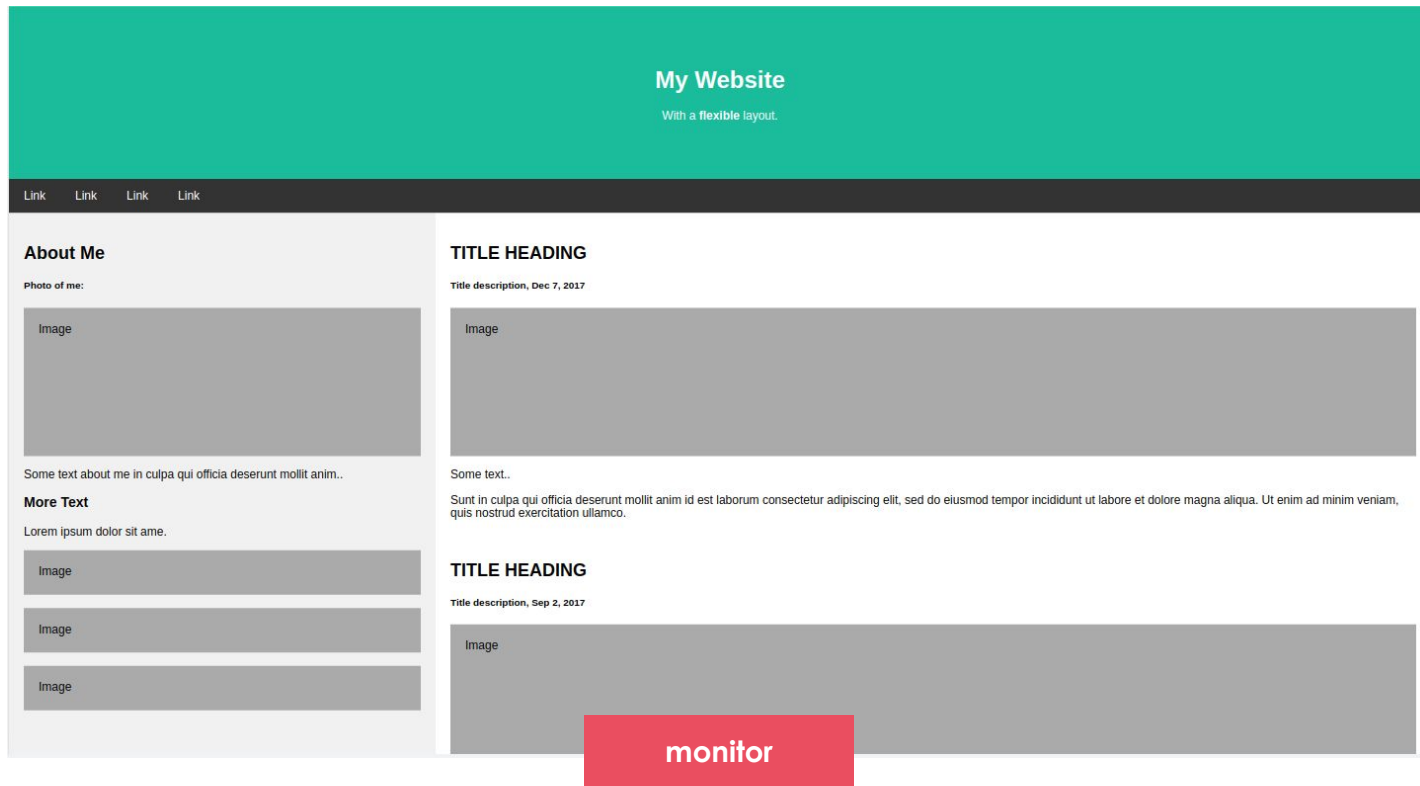
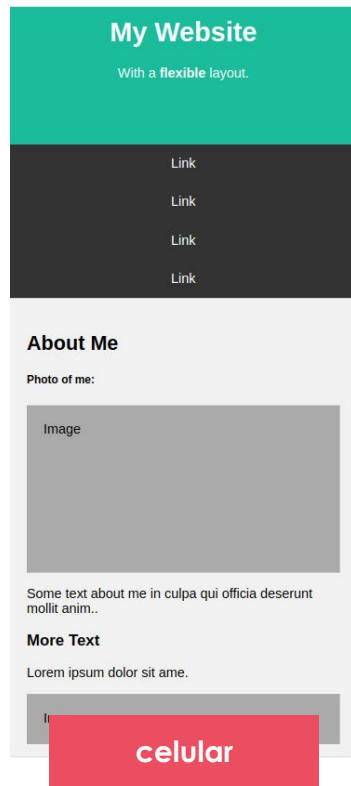
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla luctus aliquam dolor, eu lacinia lorem placerat vulputate. Duis felis orci, pulvinar id metus ut, rutrum luctus orci. Cras porttitor imperdiet nunc, at ultricies tellus laoreet sit amet. Sed auctor cursus massa at porta. Integer ligula ipsum, tristique sit amet orci vel, viverra egestas ligula. Curabitur vehicula tellus neque, ac ornare ex malesuada et. In vitae convallis lacus. Aliquam erat volutpat. Suspendisse ac imperdiet turpis. Aenean finibus sollicitudin eros pharetra congue. Duis ornare egestas augue ut luctus. Proin blandit quam nec lacus varius commodo et a urna. Ut id ornare felis, eget fermentum sapien.

celular

Flexbox

Através do flexbox, os elementos da página serão capazes de encolher ou crescer, distribuindo o espaço entre os itens de acordo com o tamanho do container.

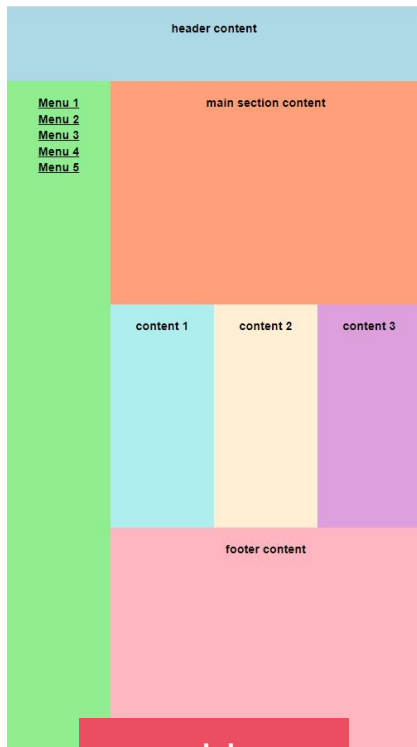
Layout flexível com flexbox



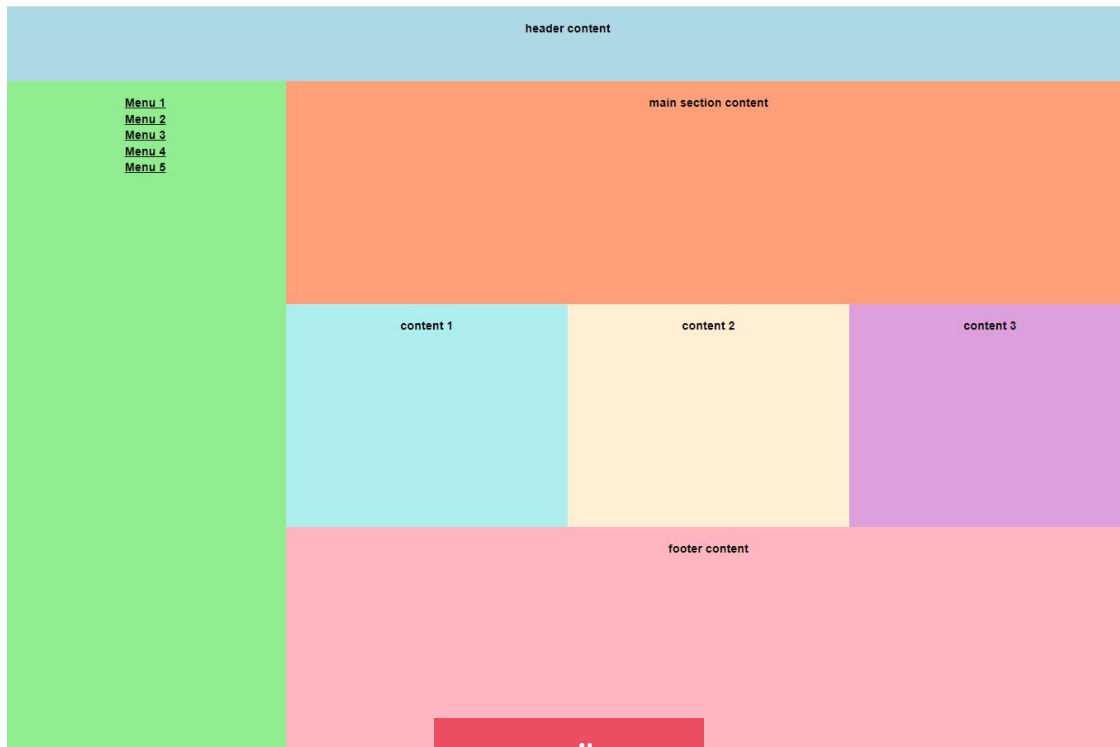
Grids flexíveis

Outro método extremamente eficaz de se criar layouts responsivos é utilizando o Grid Layout juntamente com a unidade de medida **fr**, atribuindo o espaço disponível do container entre as colunas e linhas do grid.

Layout flexível com grid



celular



monitor

Etapa 3

Meta tag viewport

Meta tag viewport

A meta tag viewport irá informar para os navegadores que eles devem utilizar a largura da janela do dispositivo para exibir a página web, fazendo com que os navegadores as redimensionem melhor, ajudando na responsividade.

```
<meta name="viewport" content="width=device-width,initial-scale=1">
```

Meta tag viewport

```
<meta name="viewport" content="width=device-width,initial-scale=1">
```

width=device-width: sobrescreve a configuração padrão dos navegadores para a largura da janela

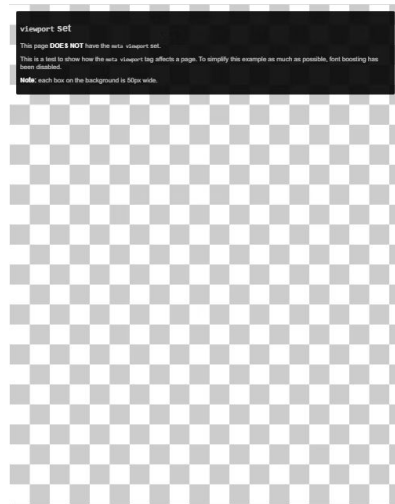
initial-scale: define zoom inicial da página

height: define altura específica para a viewport

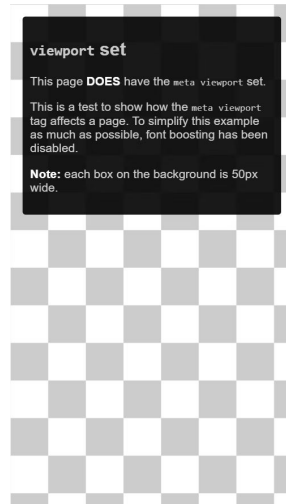
minimum-scale: define o nível mínimo de zoom

maximum-scale: define o nível máximo de zoom

user-scalable: impede que o usuário aplique zoom na página caso tenha o valor definido como **no**



sem meta tag



com meta tag



Atenção: deve-se evitar o uso do **minimum-scale**; **maximum-scale** e **user-scalable=no**

Etapa 4

O que são media queries?

Media queries

Recurso que permite aplicar propiedades do CSS somente para algumas regras de tipos de mídia específicos.

MÓVILES VERTICALES



Estilos normales

MÓVILES HORIZONTAL
TABLET VERTICAL



@media (min-width: 768px) {}

TABLET HORIZONTAL
ESCRITORIOS NORMALES



@media (min-width: 1024px) {}

ESCRITORIOS MUY ANCHOS



@media (min-width: 1200px) {}

Etapa 5

Estrutura de uma media query

Estrutura das media queries

@media screen (min-width: 320px) and (max-width: 768px) {

AT-RULE

MEDIA TYPE

MEDIA FEATURE

OPERATOR

MEDIA FEATURE

```
h1 {  
  color: red;  
}
```

ESTILOS CSS APLICADOS
PARA ESSE FILTRO

}

@media (*at-rule*)

`@media` screen (min-width: 320px) and (max-width: 768px)

AT-RULE

Regra usada para identificar o tipo de mídia que uma página está sendo visualizada, informando os recursos que esse tipo de mídia suporta e os operadores que podem ser combinados para misturar algumas condições.

@media (*at-rule*)

`@media screen (min-width: 320px) and (max-width: 768px)`

AT-RULE

Regra usada para identificar o tipo de mídia que uma página está sendo visualizada, informando os recursos que esse tipo de mídia suporta e os operadores que podem ser combinados para misturar algumas condições.

Etapa 6

Media types / tipos de mídia

media type / tipo de mídia

@media **screen** (min-width: 320px) and (max-width: 768px)

MEDIA TYPE

Define o tipo de mídia que os estilos CSS devem ser aplicados.

all: corresponde a todos os dispositivos;

print: corresponde a documentos que são uma pré-visualização de impressão, ou em qualquer mídia que será voltada para imprimir;

screen: corresponde a dispositivos com telas integradas;

speech: corresponde a dispositivos que leem o conteúdo de forma audível, como um leitor de tela, por exemplo.

Etapa 7

**Media features /
recursos de mídia**

media features / mídia recursos

@media screen (min-width: 320px) and (max-width: 768px)

MEDIA FEATURE

MEDIA FEATURE

Configura com quais recursos de mídia se quer combinar para aplicar determinados estilos.

**min-width/min-height/max-width/
max-height/width/height:** detecta a
largura e altura da viewport;

orientation: detecta se o dispositivo
está em modo retrato ou paisagem;

hover: indica que a página está sendo
acessada através de um mecanismo de
ponteiro, como um mouse;

pointer: detecta quão preciso é o
ponteiro (grosso, fino, etc), ajudando a
detectar *touchscreen*, mouse, etc.

media features: width/height

```
@media screen and (width: 600px) {  
  body {  
    color: red;  
  }  
}
```

Aplica a cor vermelha aos textos da página quando a largura da viewport for de **600px**.

media features: orientation

```
@media (orientation: landscape) {  
  body {  
    background-color: black;  
  }  
}
```

Aplica a cor de fundo preta para a página, caso a orientação do dispositivo seja paisagem (*landscape*)

media features: pointer

```
@media (pointer: coarse) {  
  input[type="checkbox"] {  
    width: 30px;  
    height: 30px;  
  }  
}
```

Redimensiona o input do tipo checkbox quando o usuário está acessando a página através de um dispositivo *touchscreen*.

fine: algo como um mouse ou trackpad;

coarse: dispositivos touchscreen;

none: o usuário não possui nenhum dispositivo de “apontamento”, ou seja, provavelmente ele está navegando com a ajuda de um teclado ou um sistema de comando de voz.

media features: e muito mais...

Existem media queries voltados para:

- Viewport/características da página;
- Qualidade da exibição;
- Cores;
- Interação;
- Vídeo;
- Script;
- Preferências do usuário.

Toda a lista também pode ser encontrada [clikando aqui](#) ou [aqui](#).



Etapa 8

Operators / Operadores

operators / operadores

@media screen (min-width: 320px) **and** (max-width: 768px)

OPERATOR

É possível combinar, através de operadores lógicos, as media features para atender as condições para aplicar os estilos

AND: deve atender a **todas** media features (recursos de mídia);

OR: deve atender, no mínimo, uma das media features (recursos de mídia);

NOT: nega as condições aplicadas, invertendo todo o significado do media query.

Etapa 9

Maneiras de usar as Media Queries

HTML, JavaScript e CSS

Além do CSS, as media queries estão presentes também através do HTML e do JavaScript. No HTML, por exemplo, podemos carregar folhas de estilos diferentes de acordo com um media query, ou aplicar na tag **<style/>**

```
<style>
p {
  background
  color: wh
}
</style>
```

```
<style media="all and (max-width: 500px)">
p {
  background-color: yellow;
  color: blue;
}
</style>
```

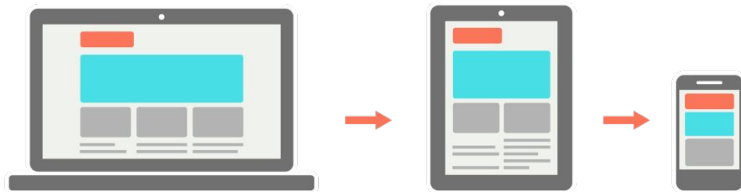
```
<html>
<head>
  <!-- Served to all users -->
  <link rel="stylesheet" href="all.css" media="all" />
  <!-- Served to screens that are at least 20em wide -->
  <link rel="stylesheet" href="small.css" media="(min-width: 320px)" />
  <!-- Served to screens that are at least 64em wide -->
  <link rel="stylesheet" href="medium.css" media="(min-width: 1024px)" />
  <!-- Served to screens that are at least 90em wide -->
  <link rel="stylesheet" href="large.css" media="(min-width: 1440px)" />
  <!-- Served to screens that are at least 120em wide -->
  <link rel="stylesheet" href="extra-large.css" media="(min-width: 1920px)" />
  <!-- Served to print media, like printers -->
  <link rel="stylesheet" href="print.css" media="print" />
</head>
<!-- ... -->
</html>
```

Etapa 10

Breakpoints / Pontos de interrupção

Breakpoints

Os *breakpoints* são os pontos de interrupção que colocamos nas media queries para tratar a quebra de layout ou ajustar algum conteúdo para determinado dispositivo para deixar as páginas responsivas.



Responsive Web Design

Breakpoint	Class infix	Dimensions
X-Small	<i>None</i>	<576px
Small	<i>sm</i>	≥576px
Medium	<i>md</i>	≥768px
Large	<i>lg</i>	≥992px
Extra large	<i>xl</i>	≥1200px
Extra extra large	<i>xxl</i>	≥1400px

Breakpoints do Bootstrap

Etapa 11

**Devemos sempre usar
as media queries?**

Design Acessível x Design Universal

“Um design acessível irá criar uma porta larga para as pessoas de cadeiras de rodas entrarem, enquanto um design universal irá produzir uma entrada onde qualquer um se entraria, desconsiderando suas habilidades.”

Laura Kalbag

Etapa 12

Imagens responsivas

Imagens responsivas

Estratégias para imagens responsivas:

1. Definir uma largura máxima com CSS;
2. Renderizar diferentes imagens para tamanhos de tela distintos (ver mais).

```
img {  
  max-width: 100%;  
  display: block;  
}
```

```

```

Etapa 13

Tipografia responsiva

Textos responsivos

Estratégias para tipografias
responsivas:

1. Unidades de medidas relativas;
2. Media Queries.

```
html {  
  font-size: 1em;  
}  
  
h1 {  
  font-size: 2rem;  
}  
  
@media (min-width: 1200px) {  
  h1 {  
    font-size: 4rem;  
  }  
}
```

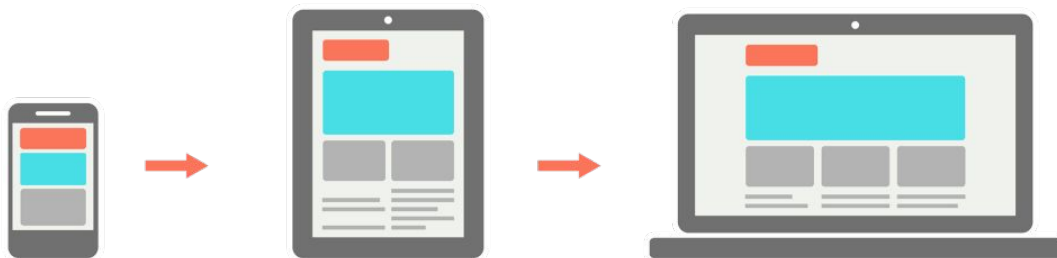
```
h1 {  
  font-size: calc(1.5rem + 3vw);  
}
```

Etapa 14

Mobile First

Mobile first

Desenvolver uma página a partir da menor viewport e ir realizando a adaptação do layout de acordo com que o tamanho da janela de visualização aumenta.



Materiais de apoio

- 🔗 Media Queries for Standard Devices
- 🔗 A Complete Guide to CSS Media Queries
- 🔗 Fundamentos de web design responsivo
- 🔗 Responsive design
- 🔗 9 basic principles of responsive web design
- 🔗 Using media queries